UNIT 3 – TREE DATA STRUCTURES

Introduction – Types of Trees – Creating a Binary Tree from a General Tree – Traversing a Binary Tree – Binary Search Trees – Operations on a Binary Search Trees – Threaded Binary Trees – AVL Trees– Binary Heaps – Applications of Heaps.

INTRODUCTION

What is a Tree?

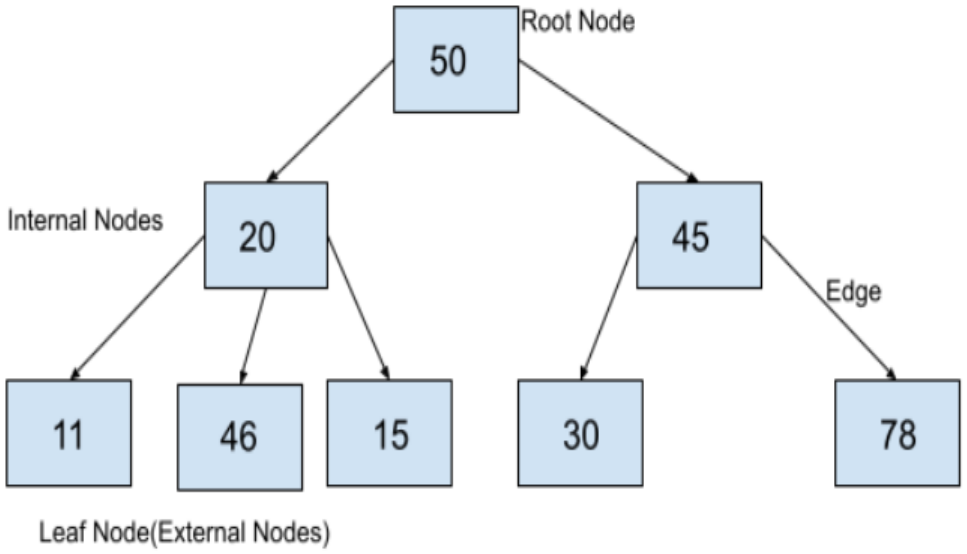
- A **tree** is a **non-linear** data structure made up of **nodes** connected by **edges**. It starts with a **root node**, and each node can have **children**. A common type of tree is the **binary tree**, where each node has at most **two children**.

Why Use Trees?

- Efficient for searching and sorting (like Binary Search Tree)
- Represent hierarchical data (e.g., file systems, organization charts)
- Backbone for complex data structures (heaps, tries, syntax trees)

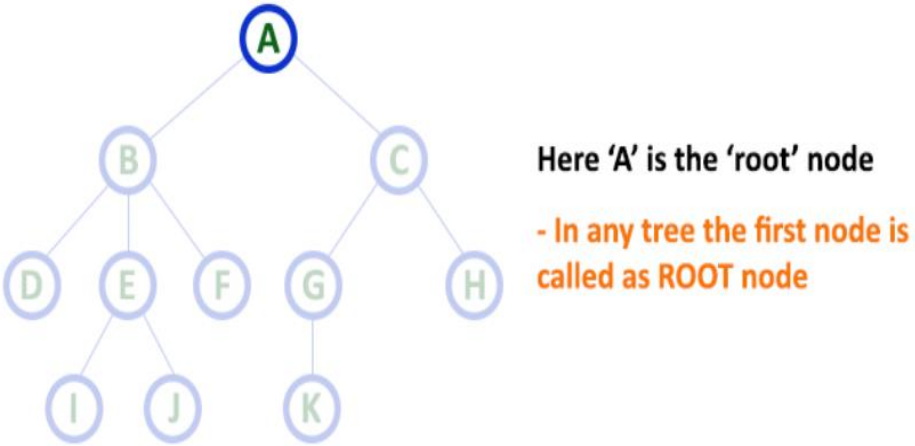
Tree ADT

- A Tree is a Data structure in which data items are connected using references in a hierarchical manner.
- **Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.**
- Each Tree consists of a root node from which can access each element of the tree.
- Starting from the root node, each node contains zero or more nodes conr

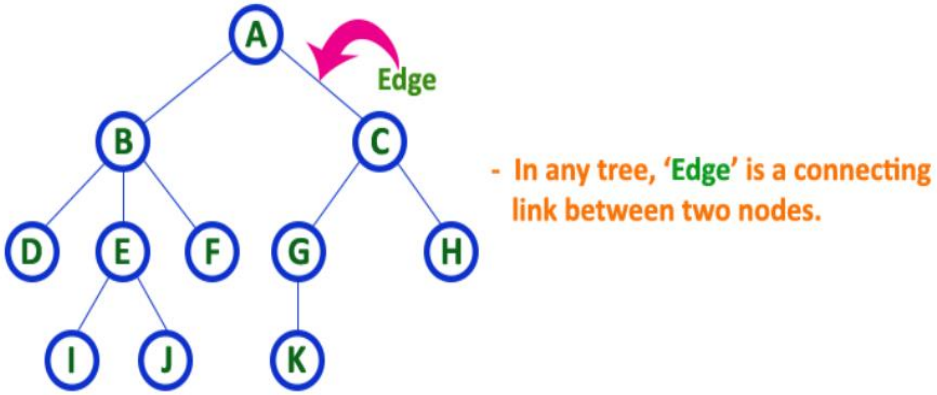


Tree Terminologies

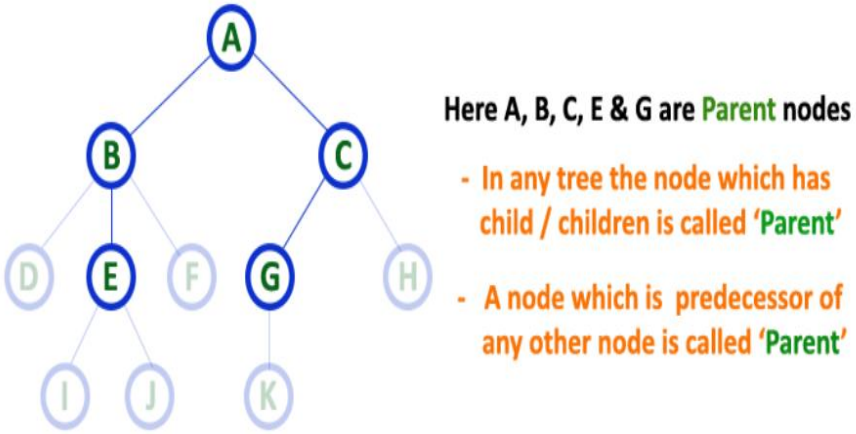
1. **Root node:** In a tree data structure, the first node is called as **Root Node**. Every tree must have a root node. The root node is the origin of the tree data structure. In



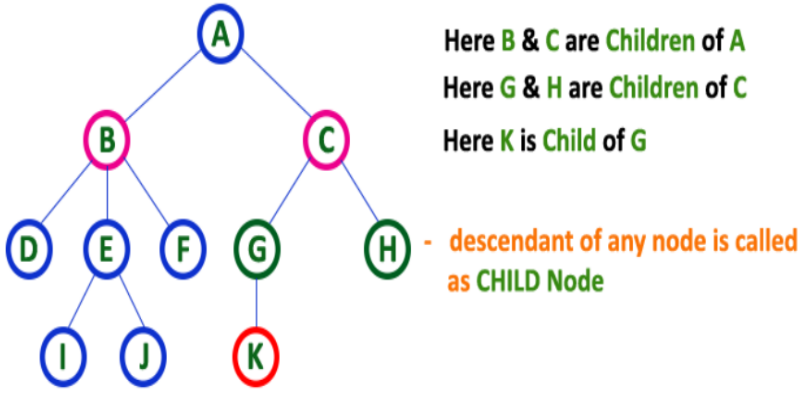
2. Edge: In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.



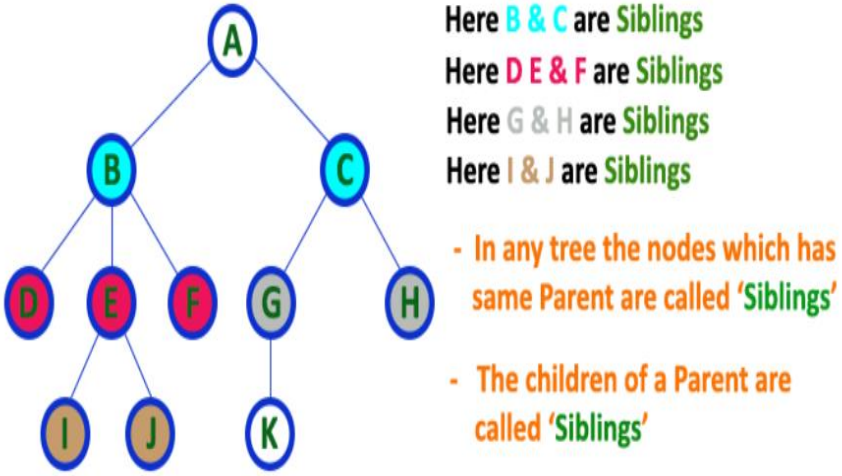
3. Parent: In a tree data structure, the node which is a predecessor of any node is called as **PARENT NODE**. The node which has a branch from it to any other node is called a parent node. Parent node can also be defined as "**The node which has child / children**".



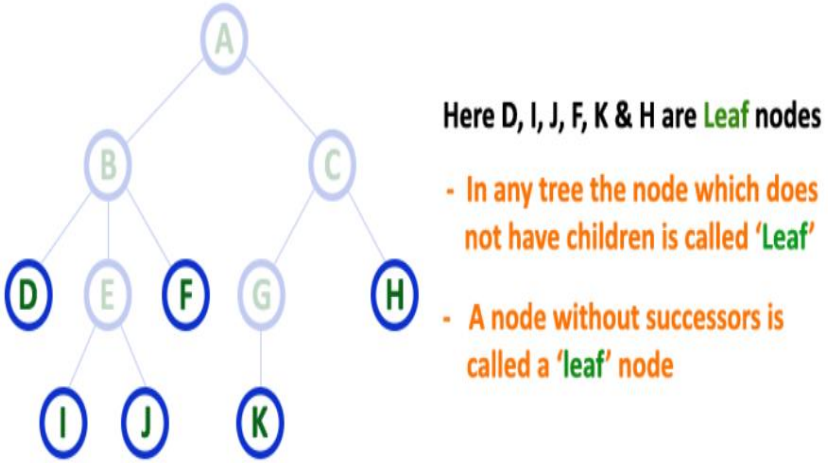
4. Child: In a tree data structure, the node which is descendant of any node is called as **CHILD Node**. The node which has a link from its parent node is called as child node. In a tree, a hild nodes. In a tree, all the nodes exce



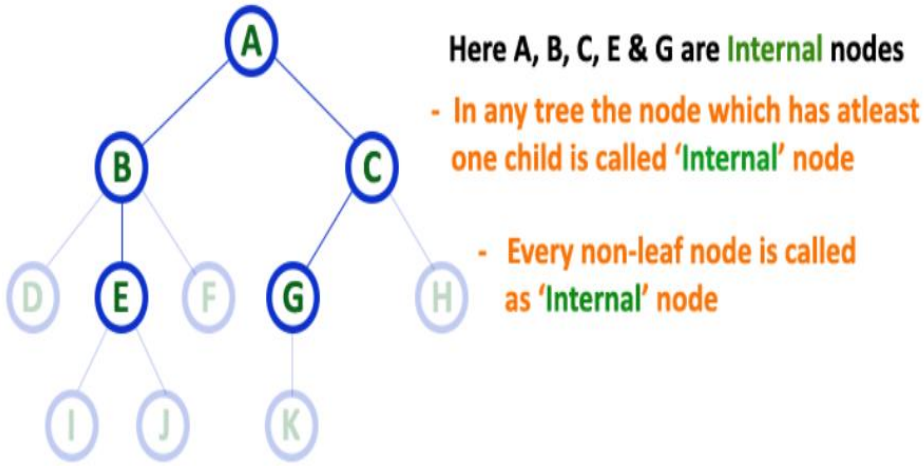
5. Siblings: In a tree data structure, nodes which belong to same Parent are called as **SIBLINGS**. The nodes with the same parent are called Sibling nodes.



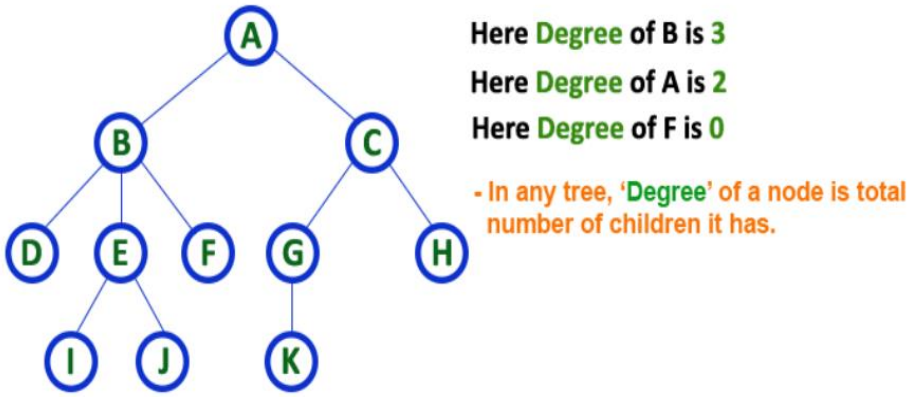
6. Leaf: In a tree data structure, the node which does not have a child is called as **LEAF Node**. The leaf nodes are also called as **External Nodes or 'Terminal'** node.



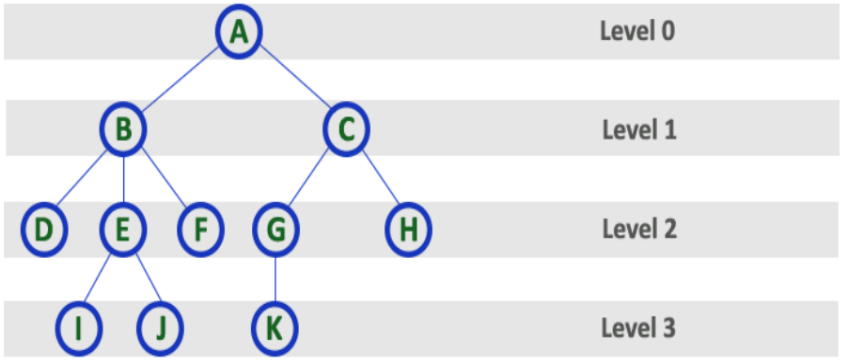
7. Internal Nodes: In a tree data structure, the node which has atleast one child is called as **INTERNAL Node**. In a tree data structure, nodes other than leaf nodes are called as **Internal Nodes or 'Non-Terminal'** nodes. **The root node is also said to be Internal Node**



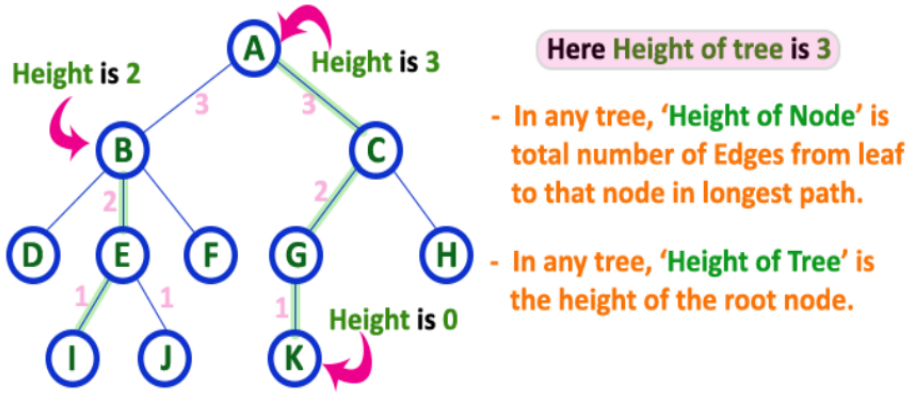
8. Degree: In a tree data structure, the total number of children of a node is called as **DEGREE** of that Node. The highest degree of a node among all the nodes in a tree is called as '**Degree of Tree**'



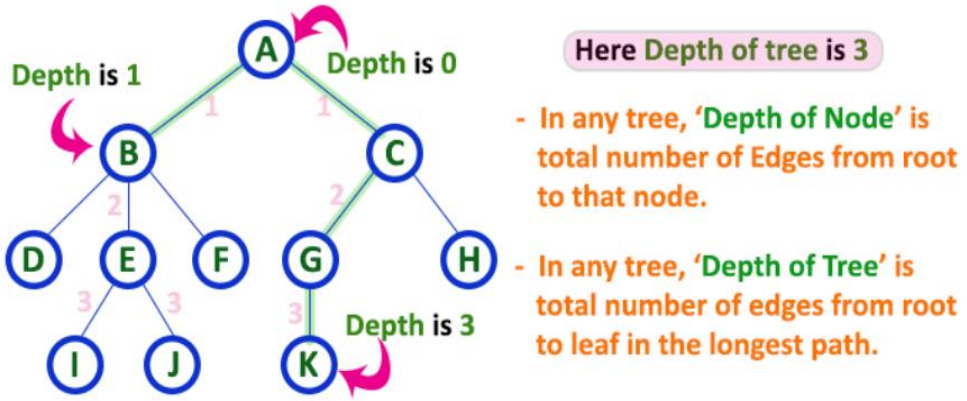
9. Level: In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).



10. Height: In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node. In a tree, height of the root node is said to be **height of the tree**. In a tree, **height of all leaf nodes is '0'.**

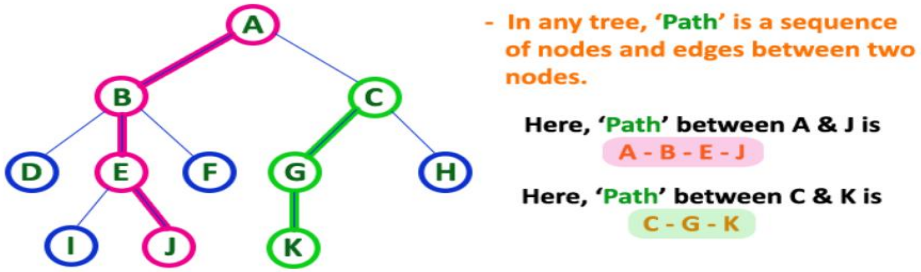


11. Depth: The total number of egdes from root node to a particular node is called as **DEPTH** of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be **Depth of the tree**. In simple words, the highest depth of any leaf node in a tree is said to be depth of that tree. In a tree, **depth of the root node is '0'.**

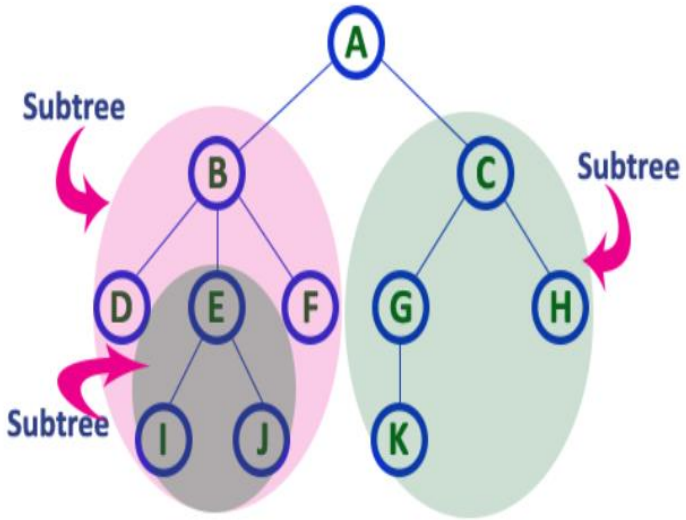


12. Path: The sequence of Nodes and Edges from one node to another node is called as **PATH** between that two Nodes. **Length of a Path** is total number of nodes in that path.

In below example **the path A - B - E - J has length 4**.



13. Sub Tree: Each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.



14. An ancestor node of a node is **either the parent of the node or the parent of some ancestor of the node**.

15. **A descendant of a node** is either a child of the node or a child of some descendant of the node.

16. *The neighbors of given node are called* ***Predecessor****(the node lies behind of given node) and* ***Successor*** *(the node lies ahead of given node).*

Types of tree

1. General Tree

- A tree where each node can have **any number of children**.
- Most flexible but complex to manage.
- Used in **file systems**, **hierarchical databases**.

2. Binary Tree

- Each node has **at most two children**: left and right.
- Basis for many advanced trees.
- **Subtypes:**
- **a. Full Binary Tree**
- Every node has **0 or 2 children**.
- No node has only one child.
- **b. Perfect Binary Tree**
- All internal nodes have 2 children.
- All leaves are at the **same level**.
- **c. Complete Binary Tree**
- All levels are completely filled **except possibly the last**, which is filled **from left to right**.
- **d. Skewed Binary Tree**
- A tree where all nodes have only one child.
  - **Left-skewed**: all children are to the left.
  - **Right-skewed**: all children are to the right.

Types of tree

3. Binary Search Tree (BST)

- A binary tree where:
  - Left child < Root
  - Right child > Root
- Great for **searching**, **inserting**, and **deleting** in O(log n) time (if balanced).

4. AVL Tree

- A self-balancing binary search tree.
- For every node, the **height difference** (balance factor) between left and right subtree is at most 1.
- Ensures better performance than unbalanced BST.

5. Red-Black Tree

- A self-balancing BST where each node has a **color** (red or black).
- Ensures balanced height for reliable performance.
- Used in STL (Standard Template Library) in C++ (e.g., map, set).

Types of tree

6. B-Tree

- A **self-balancing** tree used in **databases and file systems**.
- Designed for systems that read/write large blocks of data.
- Each node can have more than two children.

7. B+ Tree

- Extension of B-Tree.
- Used in **database indexing**.
- All data is stored in **leaf nodes**, internal nodes only act as guides.

Types of tree

8. Heap Tree

- A complete binary tree used for **priority queues**.
- **Types:**
- **Max Heap**: parent ≥ children
- **Min Heap**: parent ≤ children

9. Trie (Prefix Tree)

- A special tree used to store **strings**.
- Each path from root to leaf represents a word.
- Efficient for autocomplete and spell checking.

10. Segment Tree

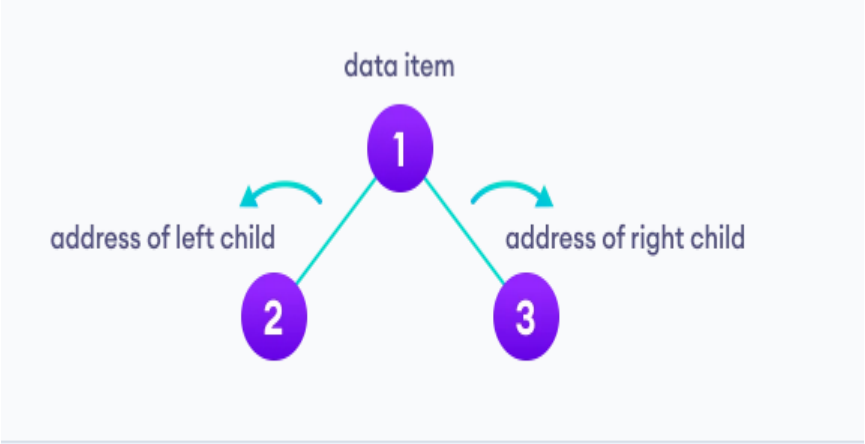
- Used for **range queries and updates** (like sum, min, max in a range).
- Common in **competitive programming**.

11. Fenwick Tree (Binary Indexed Tree)

- Also used for **range queries and updates**.
- More space-efficient and easier to implement than a segment tree (for some cases).

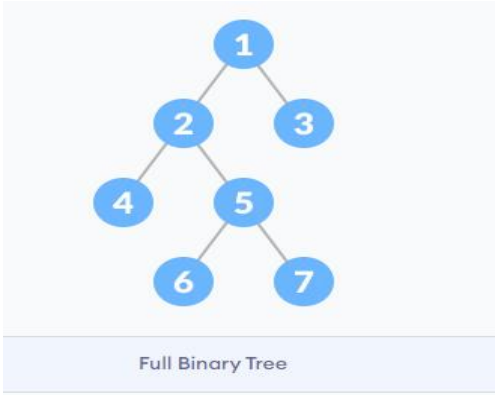
Binary tree

- A binary tree is a data structure in which every node or vertex has at most two children.
- A binary tree is a tree data structure in which each node can have a maximum of 2 children.
- Each node in a binary tree can have either one, or two or no children.
- Each node in a binary tree contains data and references to its children.
- Both the children are named as left child and the right child according to their position.
- Each node of a binary tree consists of three items: data item, address of left child, address of right child

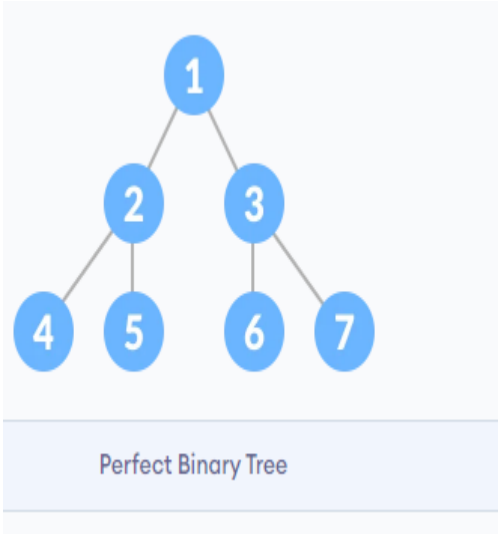


Types of Binary tree

Full Binary Tree: A full Binary tree is a special type of binary tree in which every parent node/internal node has either two or no children.

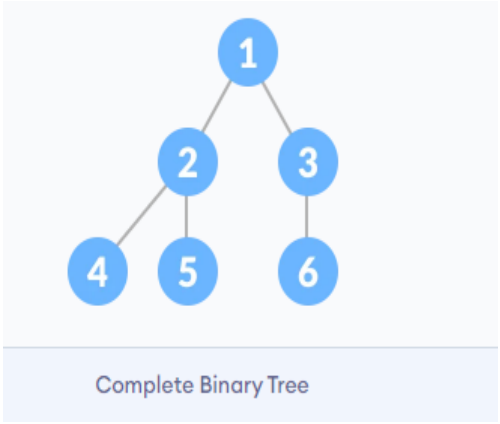


Perfect Binary Tree: A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level**.**



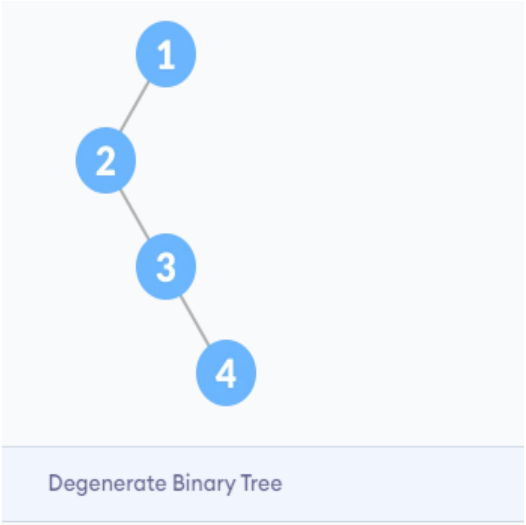
Complete Binary Tree: A complete binary tree is just like a full binary tree, but with two major differences

- Every level must be completely filled
- All the leaf elements must lean towards the left.
- The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.



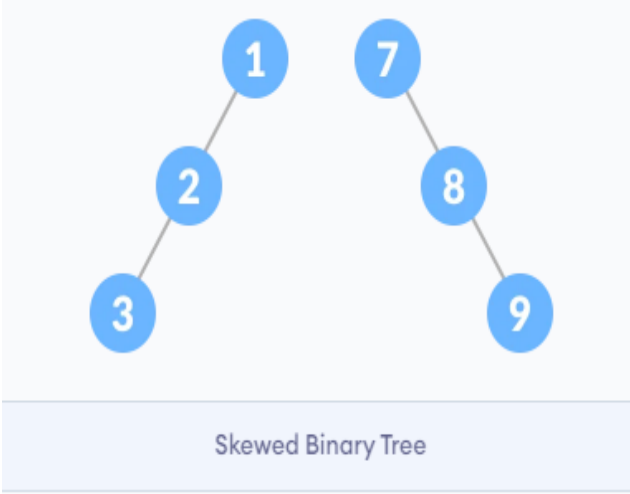
Complete Binary Tree

Degenerate or Pathological Tree: A degenerate or pathological tree is the tree having a single child either left or right.



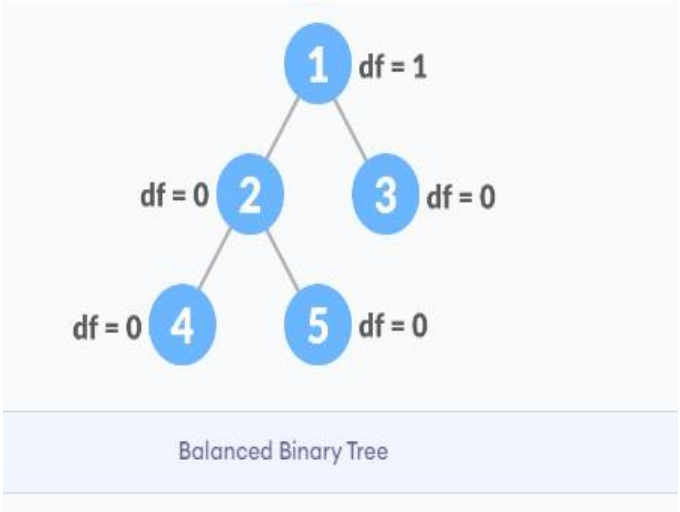
Degenerate Binary Tree

Skewed Binary Tree: A skewed binary tree is a pathological/degenerate tree in which the tree is either dominated by the left nodes or the right nodes. Thus, there are two types of skewed binary tree: **left-skewed binary tree** and **right-skewed binary tree**.



Skewed Binary Tree

Balanced Binary Tree: It is a type of binary tree in which the difference between the height of the left and the right subtree for each node is either 0 or 1.



Balanced Binary Tree

Properties of Binary Trees

1. Maximum Nodes at Level 'l'

- A binary tree can have at most $\mathbf{2^l}$ nodes at level **l**.
- **Level Definition:** The number of edges in the path from the root to a node. The root is at level **0**.
- **Proof by Induction:**
- **Base case:** For root (**l = 0**), nodes = $2^0 = 1$.
- **Inductive step:** If level **l** has 2^l nodes, then the next level has at most **twice** as many:
- $\mathbf{2 \times 2^l = 2^{l+1}}$

2. Maximum Nodes in a Binary Tree of Height 'h'

- A binary tree of height **h** can have at most $\mathbf{2^{h+1} - 1}$ nodes.
- **Height Definition:** The longest path from the root to a leaf node. Please note that a tree with only one root node is considered to have height 0 and an empty tree (or root is NULL) is considered to have height "-1"
- **Formula Derivation:** A tree has the **maximum nodes when all levels are completely filled**. Summing nodes at each level:
- $\mathbf{1 + 2 + 4 + \ldots + 2^h = 2^{h+1} - 1}$
- **Alternate Height Convention:** Some books consider a tree with only one root node is considered to have height 1 and an empty tree (or root is NULL) is considered to have height 0. making the formula $\mathbf{2^h - 1.}$

Properties of Binary Trees

3. Minimum Height for 'N' Nodes

- The minimum possible height for **N** nodes is $\lfloor \log_2 N \rfloor$.
- **Explanation:** A binary tree with height **h** can have at most $2^{h+1} - 1$ nodes.
- Rearranging:
- $N \leq 2^{h+1} - 1$
  $2^{h+1} \geq N+1$
  $h \geq \log_2(N+1) - 1$ *(Taking $\log_2$ both sides)*
  $h \geq \lfloor \log_2 N \rfloor$
- This means a binary tree with **N nodes must have at least $\lfloor \log_2 N \rfloor$ levels**.

4. Minimum Levels for 'L' Leaves

- A binary tree with **L** leaves must have at least $\lfloor \log_2 L \rfloor$ levels.
- **Why?** A tree has the **maximum number of leaves when all levels are fully filled**.
- **From Property 1:**
  $L \leq 2^l$ (**l** is the level where leaves appear)
- Solving for **l**:
  $l_{min} = \lfloor \log_2 L \rfloor$
- This gives the **minimum levels needed to accommodate L leaves**.

Properties of Binary Trees

5. Nodes with Two Children vs. Leaf Nodes

- In a **full binary tree** (where every node has either 0 or 2 children), the number of **leaf nodes (L) is always one more than the internal nodes (T) with two children**:
- L=T+1
- **Proof:**
- A full binary tree has a total of $2^{h+1} - 1$ nodes.
- Leaves are at the last level: $L = 2^h$.
- Internal nodes: $T = 2^h (2-1) - 1 = 2^h - 1$.
- Simplifies to **L=T+1**

6. Total Edges in a Binary Tree

- In any **non-empty binary tree** with **n** nodes, the total number of edges is **n - 1**.
- Every node (except the root) has exactly **one parent**, and each parent-child connection represents an **edge**.
- Since there are **n nodes**, there must be **n - 1 edges**.

Binary Tree Properties-

- Minimum number of nodes in a binary tree of height H = H + 1
- Maximum number of nodes in a binary tree of height H = $2^{H+1} - 1$
- Total Number of leaf nodes in a Binary Tree = Total Number of nodes with 2 children + 1
- Maximum number of nodes at any level ‘L’ in a binary tree = 2^{L}

```
#include<stdio.h>
#include<stdlib.h>
// We are creating struct for the binary tree below
struct node
{
  int data;
  struct node *left, *right;
};

// newNode function for initialisation of the newly created node
struct node *newNode (int item)
{
  struct node *temporary = (struct node *) malloc (sizeof (struct node));
  temporary->data = item;
  temporary->left = temporary->right = NULL;
  return temporary;
```

Tree Traversal

Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree. There are three ways which we use to traverse a tree −

Preorder traversal

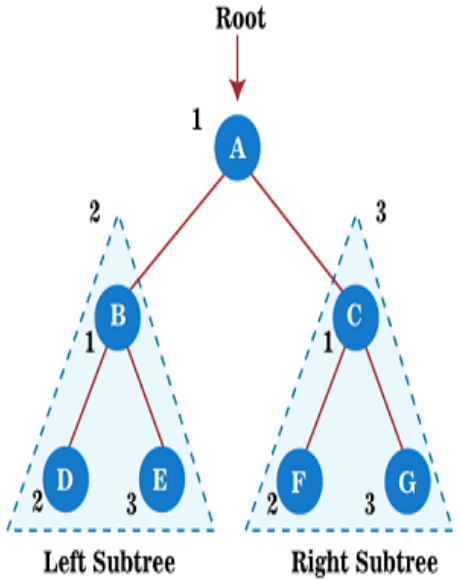
This technique follows the 'root left right' policy. It means that, first root node is visited after that the left subtree is traversed recursively, and finally, right subtree is recursively traversed. As the root node is traversed before (or pre) the left and right subtree, it is called preorder traversal.

The applications of preorder traversal include -

- It is used to create a copy of the tree.
- It can also be used to get the prefix expression of an e

Algorithm

- Step 1 - Visit the root node
- Step 2 - Traverse the left subtree recursively.
- Step 3 - Traverse the right subtree recursively.

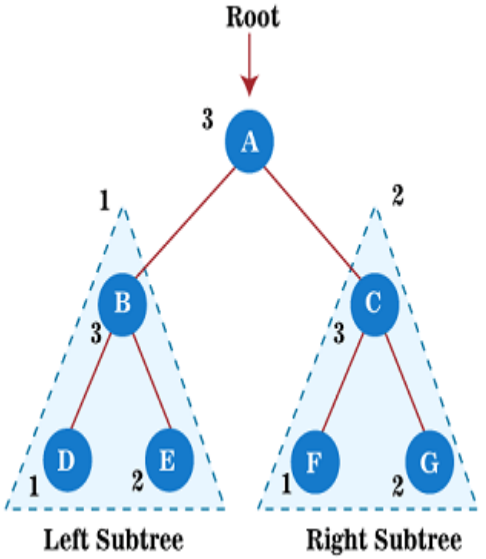


→ D → E → C → F → G

```
void preorder (struct node *root)
{
  if (root != NULL)
    {
      printf ("%d ", root->data);
      preorder (root->left);
      preorder (root->right);
```

Postorder traversal

This technique follows the 'left-right root' policy. It means that the first left subtree of the root node is traversed, after that recursively traverses the right subtree, and finally, the root node is traversed. As the root node is traversed after (or post) the left and right subtree, it is called postorder traversal.



D → E → B → F → G → C → A

The applications of postorder traversal include –

- It is used to delete the tree.
- It can also be used to get the postfix expression of a

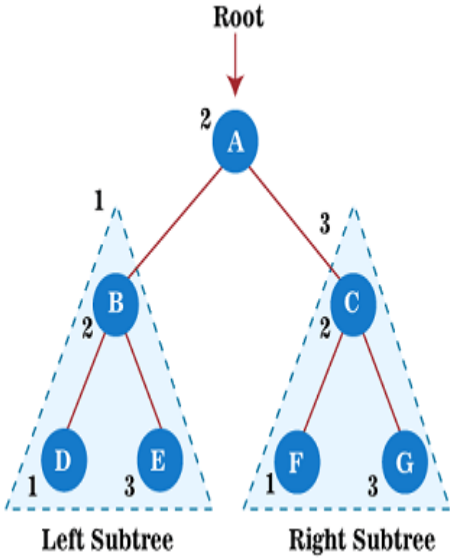
Algorithm

- Step 1 - Traverse the left subtree recursively.
- Step 2 - Traverse the right subtree recursively.
- Step 3 - Visit the root node.

```
void postorder (struct node *root)
{
  if (root != NULL)
     {
       postorder (root->left);
       postorder (root->right);
       printf ("%d ", root->data);
     }
}
```

Inorder traversal

This technique follows the 'left root right' policy. It means that first left subtree is visited after that root node is traversed, and finally, the right subtree is traversed. As the root node is traversed between the left and right subtree, it is named inorder traversal.



The applications of Inorder traversal includes -

- It is used to get the BST nodes in increasing order.
- It can also be used to get the prefix expression of a

Algorithm

- Step 1 - Traverse the left subtree recursively.
- Step 2 - Visit the root node.
- Step 3 - Traverse the right subtree recursively.

D → B → E → A → F → C → G

```
void inorder (struct node *root)
{
  if (root != NULL)
     {
      inorder (root->left);
      printf ("%d ", root->data);
      inorder (root->right);
     }
}
```

Binary search tree

What is a Binary Search Tree?

- A **Binary Search Tree (BST)** is a binary tree with the following properties:
  - Each node has **at most two children**
  - The **left child** contains values **less than** the parent
  - The **right child** contains values **greater than** the parent
- BSTs allow **efficient search, insertion, and deletion**

Why Use BSTs?

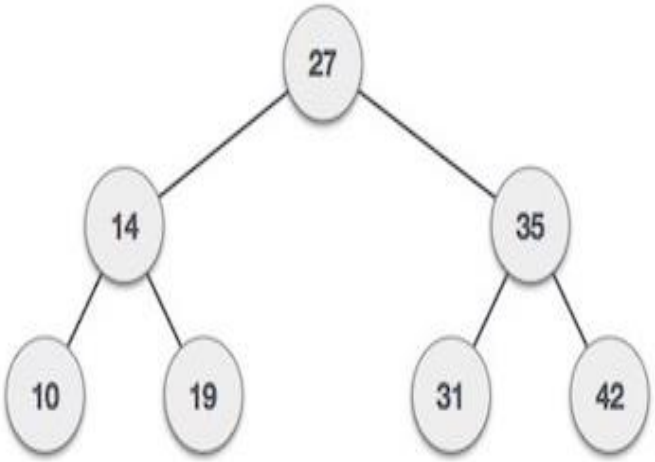
- Efficient **search operations**: O(log n) on average
- Maintains **sorted data**
- Useful in implementing:
  - Maps and Sets
  - Database indices
  - Memory management

Binary Search Tree

A Binary Search Tree (BST) is a tree in which all the nodes follow the below mentioned properties −

- The value of the key of the left sub-tree is less than the value of its parent (root) node's key.
- The value of the key of the right sub-tree is greater than or equal to the value of its parent (root) node's key.
- BST divides all its sub-trees into two segments; the left sub-tree and the r

```
left_subtree (keys) < node (key) ≤ right_subtree (keys)
```



BST Node Structure in C

```
struct Node {
int data;
struct Node* left; struct Node* right;
};
```

- Each node contains:
  - An integer value
  - Pointer to the left child
  - Pointer to the right child

Basic Operations

Following are the basic operations of a tree −

- **Search** − Searches an element in a tree.
- **Insert** − Inserts an element in a tree.
- **Delete** – remove a element from tree
- **Pre-order Traversal** − Traverses a tree in a pre-order manner.
- **In-order Traversal** − Traverses a tree in an in-order manner.
- **Post-order Traversal** − Traverses a tree in a post-order manner.

Advantages of Binary search tree

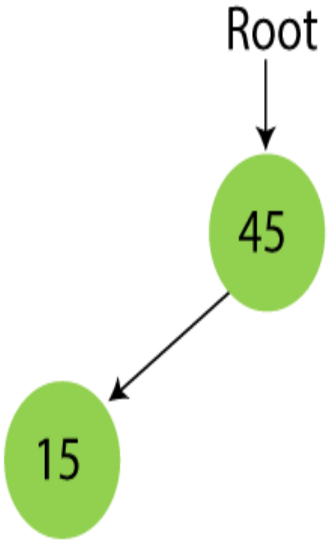
- Searching an element in the Binary search tree is easy as we always have a hint that which sub tree has the desired element.
- As compared to array and linked lists, insertion and deletion operations are faster in BST.

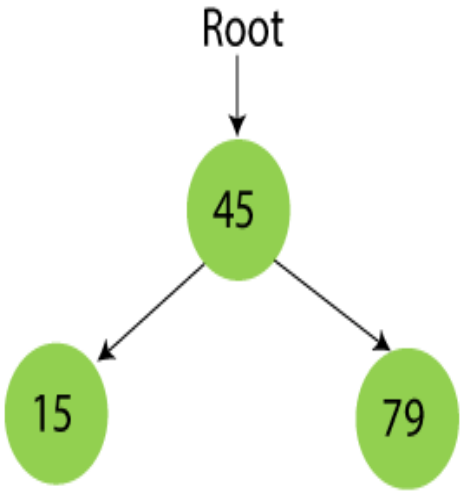
Example

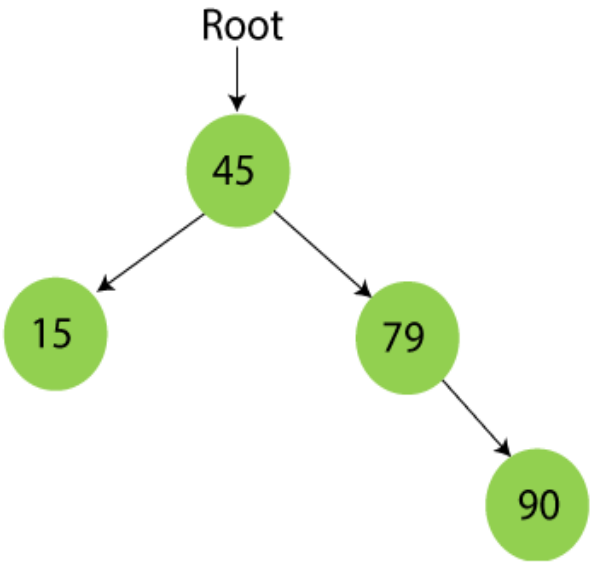
Suppose the data elements are **- 45, 15, 79, 90, 10, 55, 12, 20, 50**

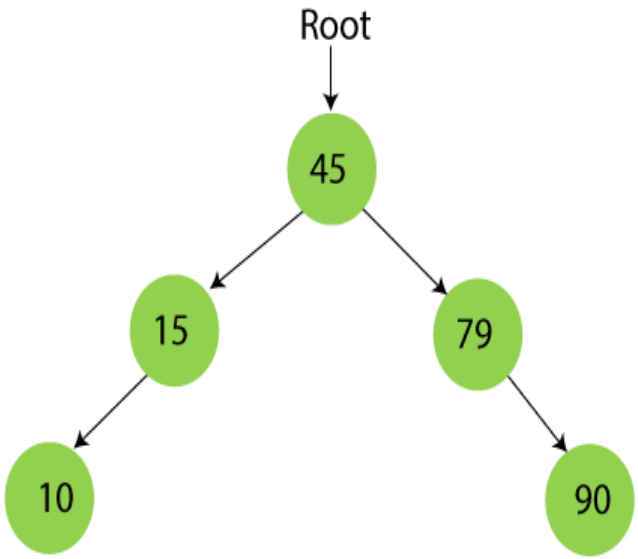
- First, we have to insert **45** into the tree as the root of the tree.
- Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.
- Otherwise, if the element is larger than the root node, then insert it as the root of t

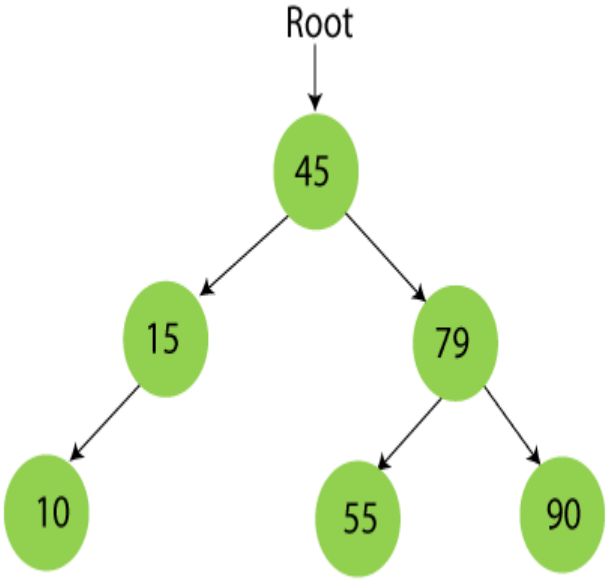


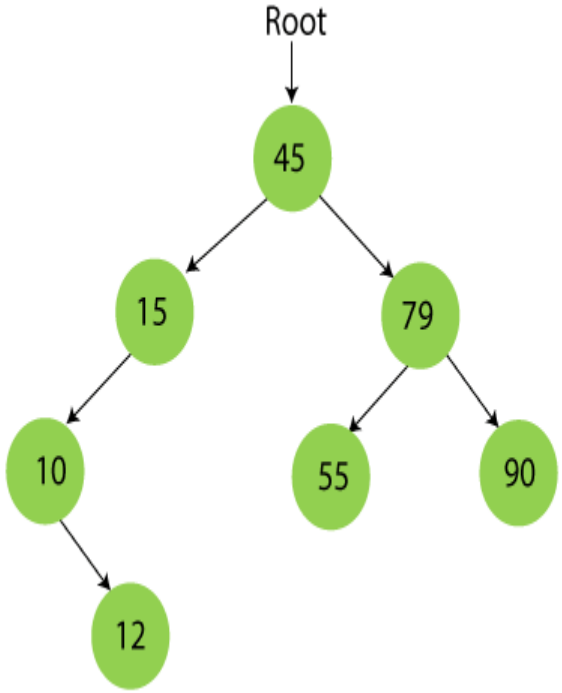


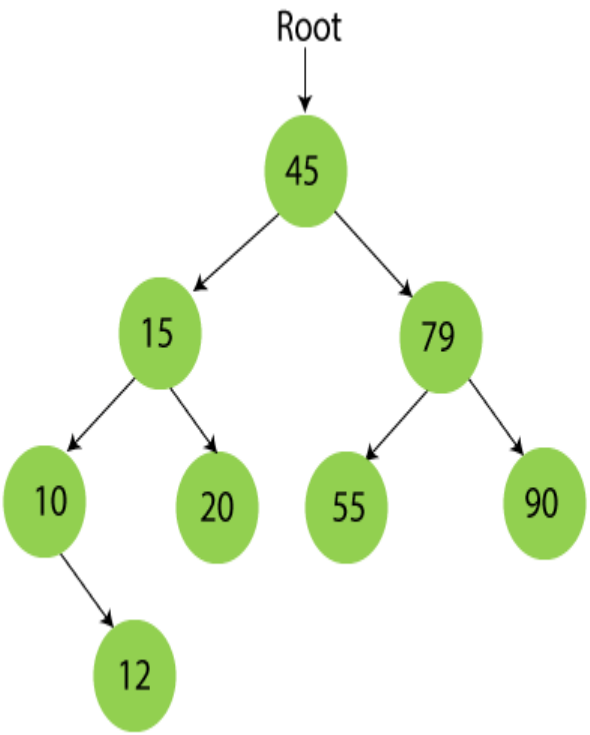


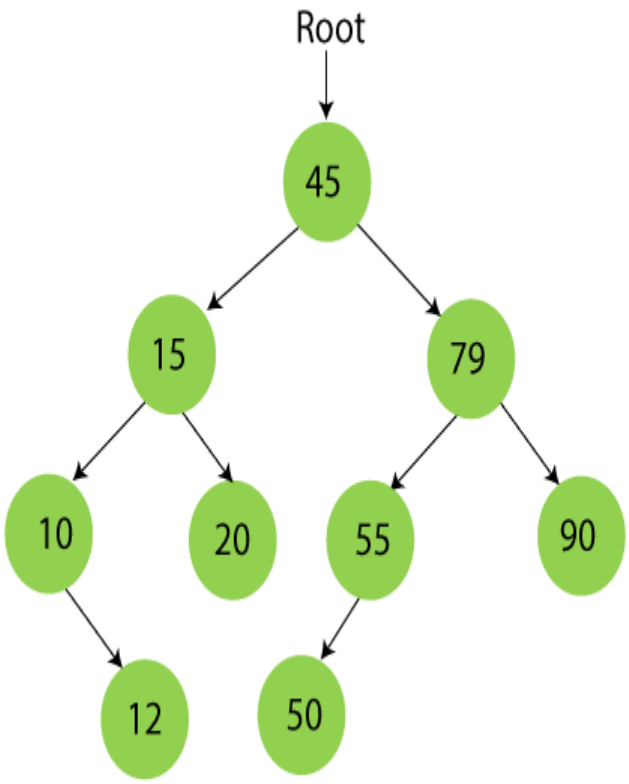

Root
45
15
79
90


Root
45
15
79
10
90


Root
45
15
79
10
55
90


Root
45
15
79
10
55
90
12


Root
45
15
79
10
20
55
90
12


Root
45
15
79
10
20
55
90
12
50

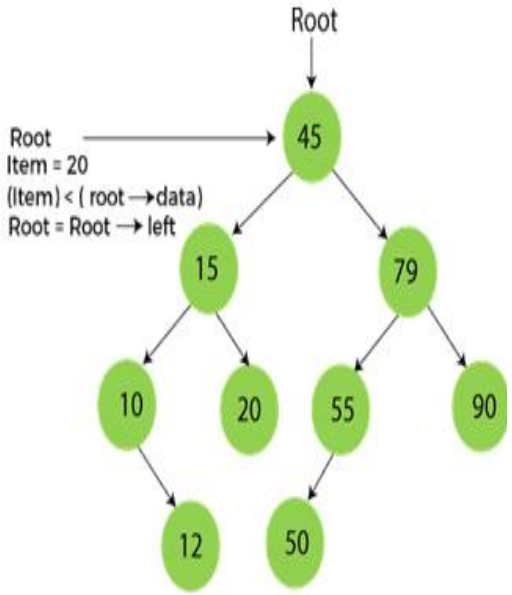
INSERTION ON BST

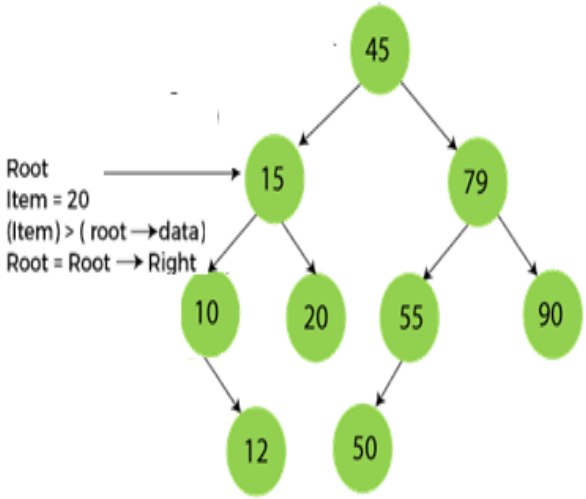
- Start from the root
- If the new value is sma
- If larger, go right
- Repeat until a NULL ch

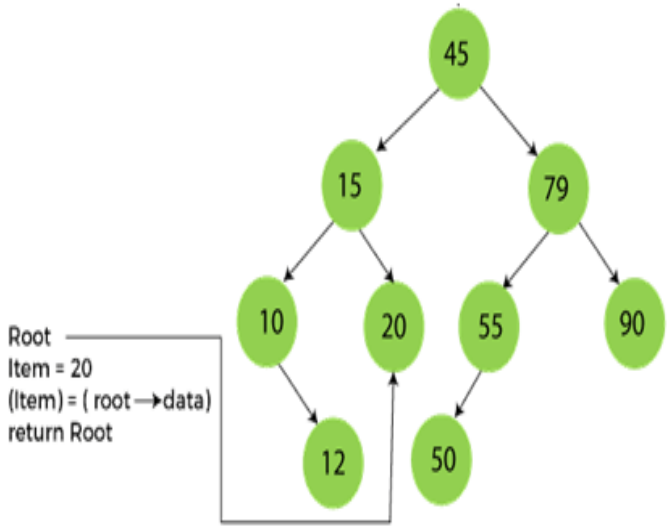
```
struct Node* insert(struct Node* root, int key) {
    if (root == NULL) {
        root = (struct Node*)malloc(sizeof(struct Node));
        root->data = key;
        root->left = root->right = NULL;
    } else if (key < root->data)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);
    return root;
}
```

Searching in Binary search tree

- First, compare the element to be searched with the root element of the tree.
- If root is matched with the target element, then return the node's location.
- If it is not matched, then check whether the item is less than the root element, if it is smaller than the root element, then move to the left subtree.
- If it is larger than the root element, then move to the right subtree.
- Repeat the above procedure recursively until the match is found.
- If the element is not found or not present in the tree, then return NULL.







SEARCH IN BST

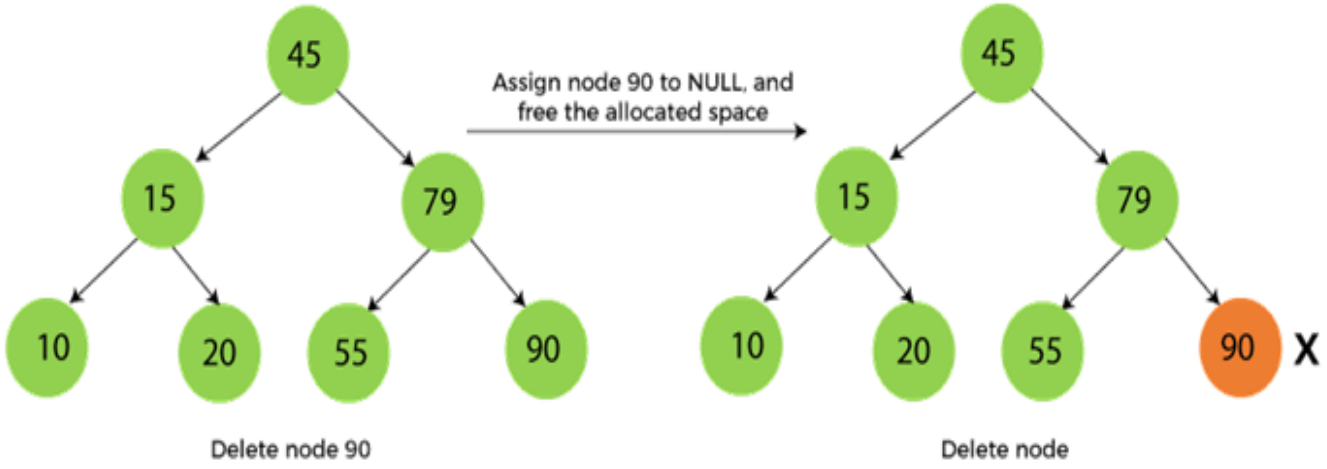
```
struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->data == key)
        return root;
    if (key < root->data)
        return search(root->left, key);
    return search(root->right, key);
}
```

Deletion in Binary Search tree

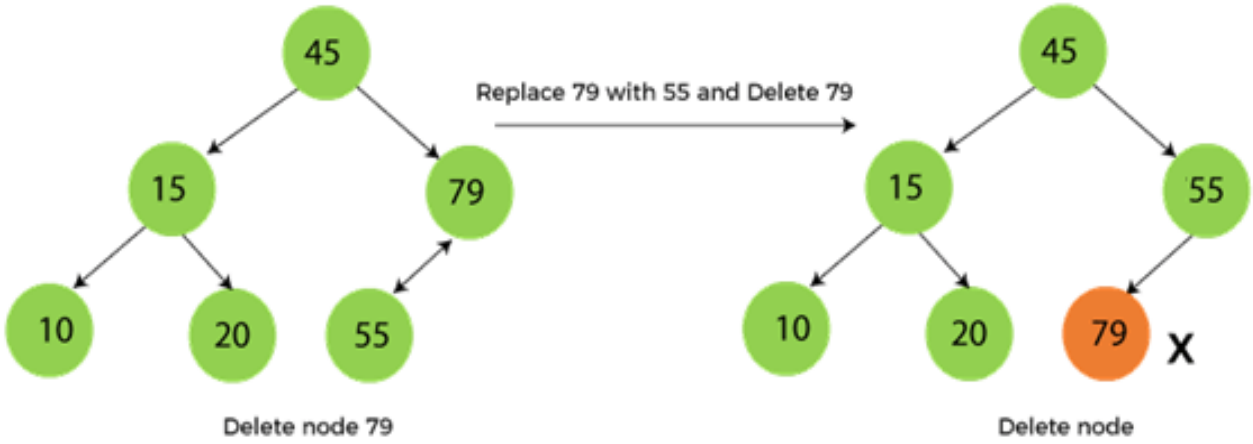
In a binary search tree, we must delete a node from the tree by keeping in mind that the property of BST is not violated. To delete a node from BST, there are three possible situations occur -

- The node to be deleted is the leaf node, or,
- The node to be deleted has only one child, and,
- The node to be deleted has two children

When the node to be deleted is the leaf node, to replace the leaf node with NULL and simply free the allocated space.



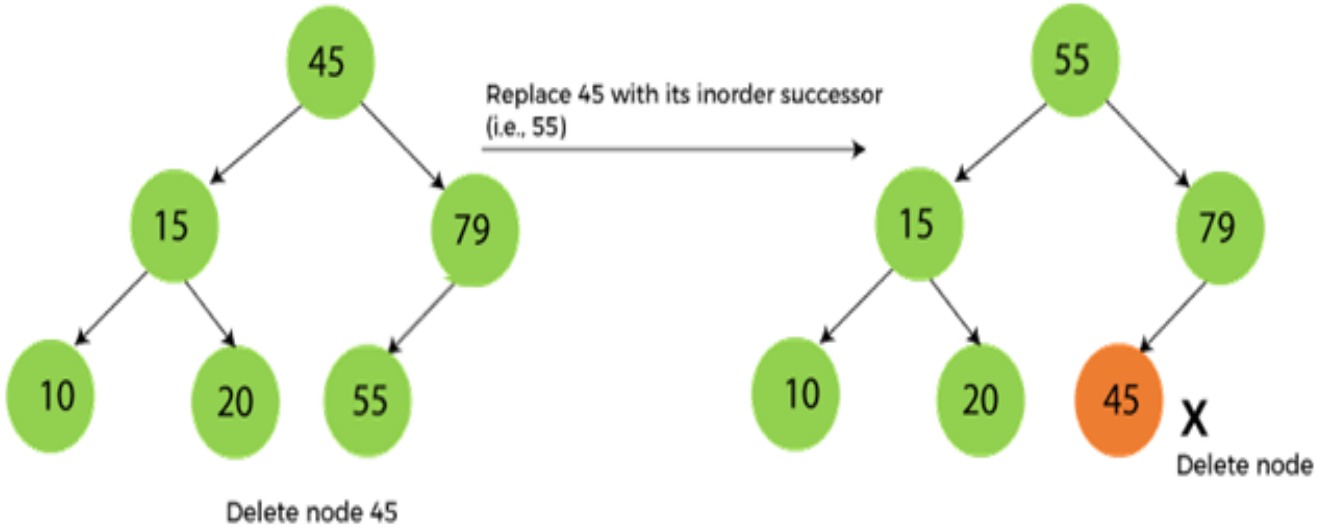
When the node to be deleted has only one child, to replace the target node with its child, and then delete the child node.



When the node to be deleted has two children

This case of deleting a node in BST is a bit complex among other two cases. In such a case, the steps to be followed are listed as follows -

- First, find the inorder successor of the node to be deleted.
- After that, replace that node with the inorder successor until the target node is placed at the leaf of tree.
- And at last, replace the node with NULL and free up the allocated space.



```
struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return root;
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Node with only one child or no child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
```

```
    // Node with two children: get the inorder successor
    struct Node* temp = findMin(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);

}

return root;
```

```
// Algorithm: Find minimum node (used in delete)
struct Node* findMin(struct Node* root) {
    while (root && root->left != NULL)
        root = root->left;

    return root;
}
```

AVL tree

AVL tree is a binary search tree in which the difference of heights of left and right subtrees of any node is less than or equal to one. The technique of balancing the height of binary trees was developed by Adelson, Velskii, and Landi and hence given the short form as AVL tree or Balanced Binary Tree.

An AVL tree can be defined as follows:

- Let T be a non-empty binary tree with T_L and T_R as its left and right subtrees. The tree is height balanced if:
- T_L and T_R are height balanced
- $h_L - h_R <= 1$, where $h_L - h_R$ are the heights of T_L and T_R

The Balance factor of a node in a binary tree can have value 1, -1, 0, depending on whether the height of its left subtree is greater, less than or equal to the height of the right subtree.

Why AVL Tree?

AVL tree controls the height of the binary search tree by not letting it to be skewed. The time taken for all operations in a binary search tree of

Operations on AVL tree

SN	Operation	Description
1	Insertion	Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.
2	Deletion	Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

Complexity

Algorithm	Average case	Worst case
Space	o(n)	o(n)
Search	o(log n)	o(log n)
Insert	o(log n)	o(log n)
Delete	o(log n)	o(log n)

AVL Rotations

We perform rotation in AVL tree only in case if Balance Factor is other than -1, 0, and 1.

There are basically four types of rotations which are as follows:

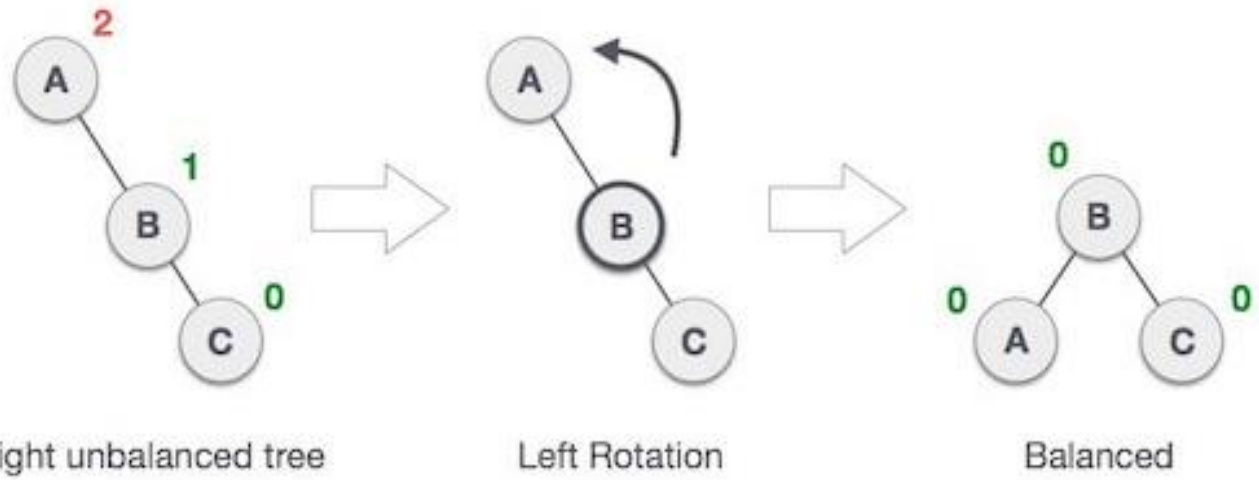
L L rotation: Inserted node is in the left subtree of left subtree of A

R R rotation : Inserted node is in the right subtree of right subtree of A

L R rotation : Inserted node is in the right subtree of left subtree of A

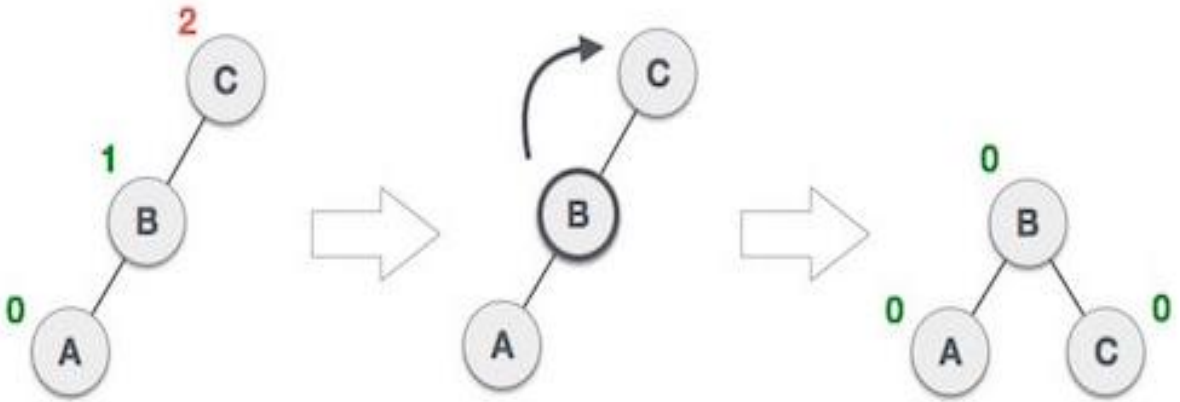
R L rotation : Inserted node is in the left subtree of right subtree of A

1. RR Rotation



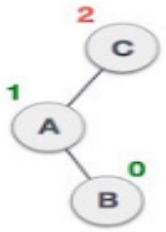
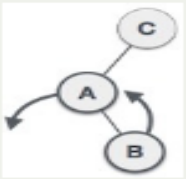
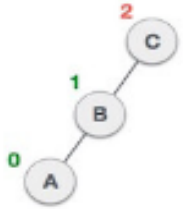
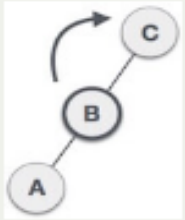
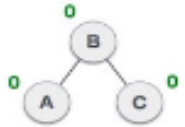
Right unbalanced tree | Left Rotation | Balanced

2. LL Rotation

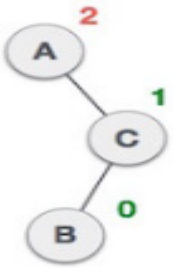
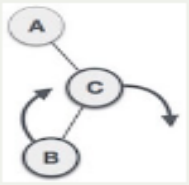
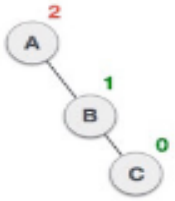
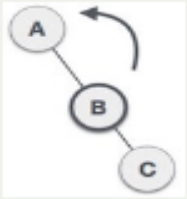
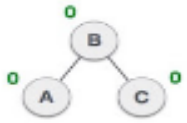


Left unbalanced Tree | Right Rotation | Balanced Tree

3. LR rotation

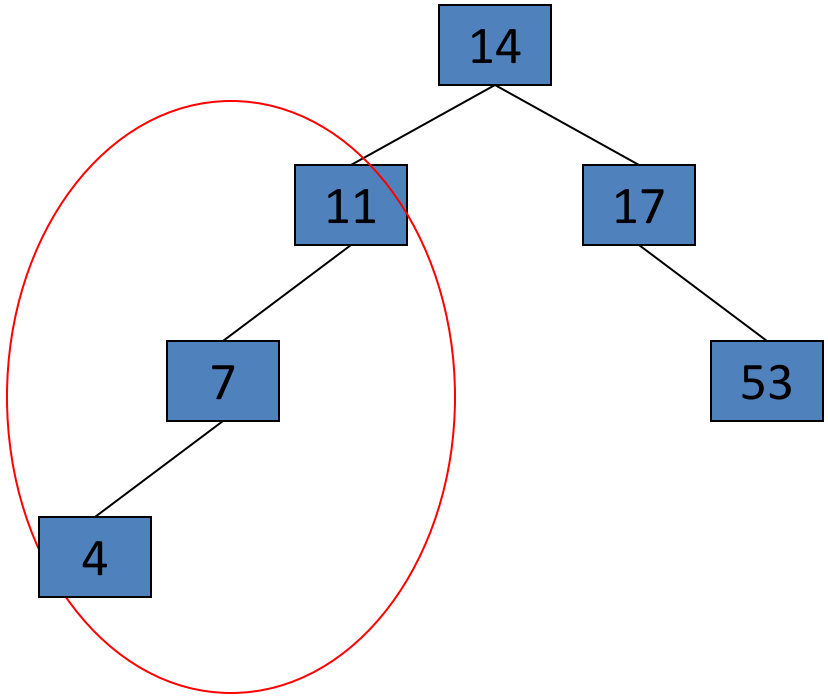
State	Action
	A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C
	As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node **A**, has become the left subtree of **B**.
	After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of **C**
	Now we perform LL clockwise rotation on full tree, i.e. on node C. node **C** has now become the right subtree of node B, A is left subtree of B
	Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

4. RL Rotation

State	Action
	A node **B** has been inserted into the left subtree of **C** the right subtree of **A**, because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A
	As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at **C** is performed first. By doing RR rotation, node **C** has become the right subtree of **B**.
	After performing LL rotation, node **A** is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node A.
	Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A. node **C** has now become the right subtree of node B, and node A has become the left subtree of B.
	Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.

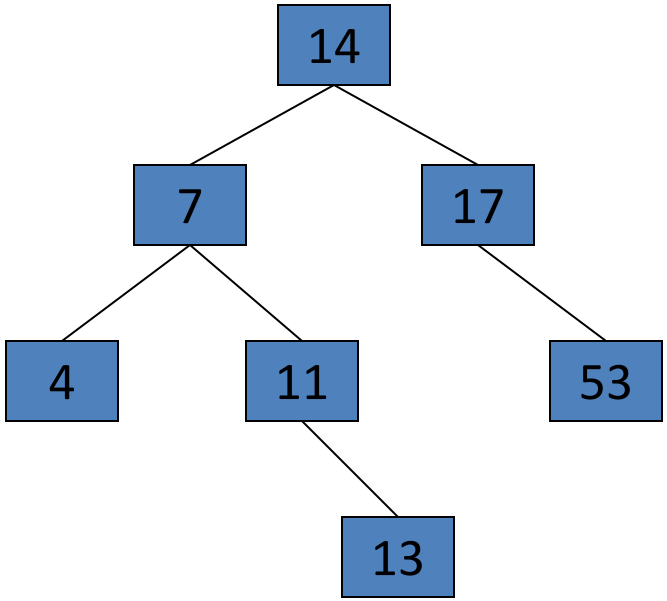
AVL Tree Example:

- **Insert 14, 17, 11, 7, 53, 4, 13 into an empty AVL tree**



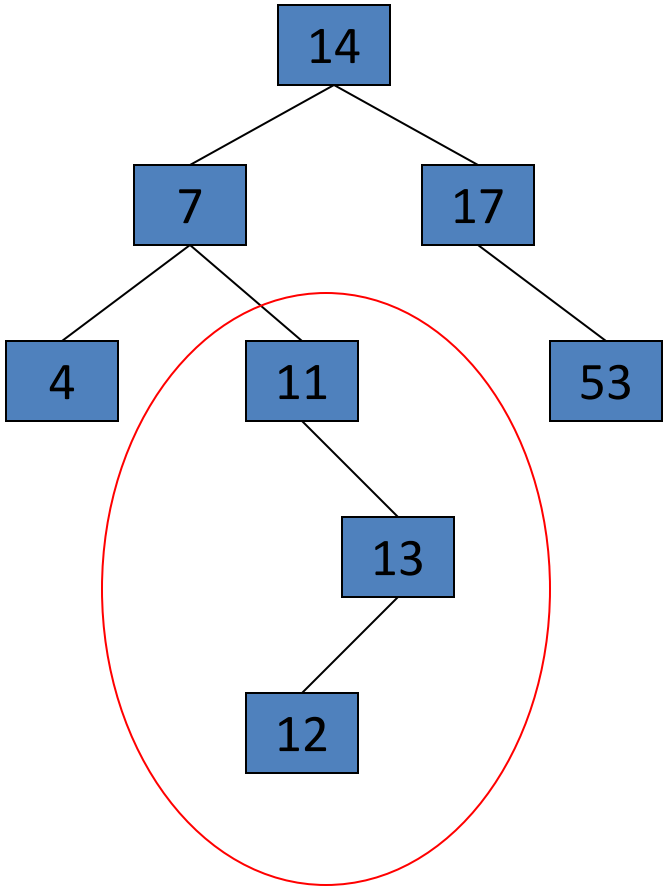
AVL Tree Example:

- **Insert 14, 17, 11, 7, 53, 4, 13 into an empty AVL tree**



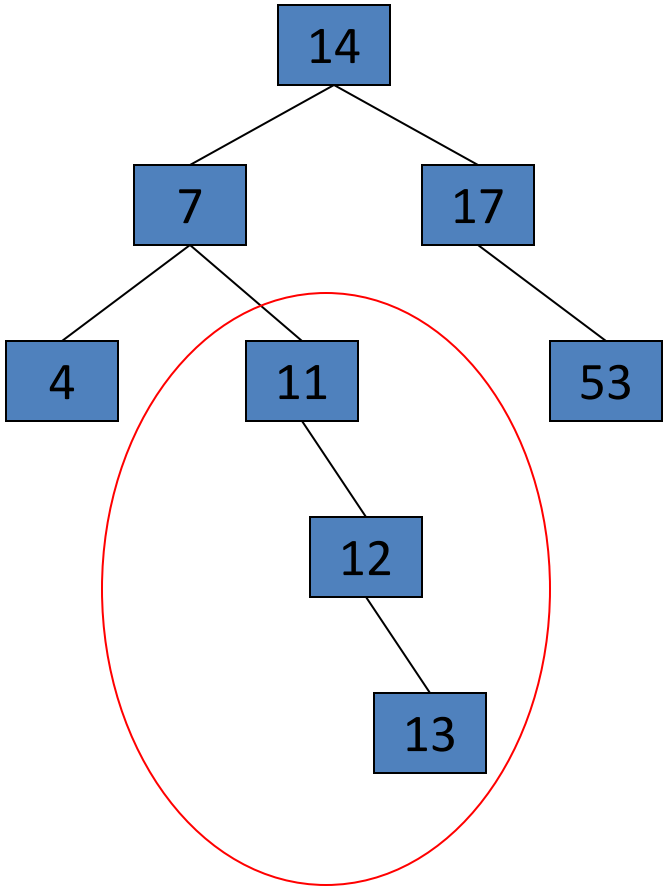
AVL Tree Example:

- **Now insert 12**



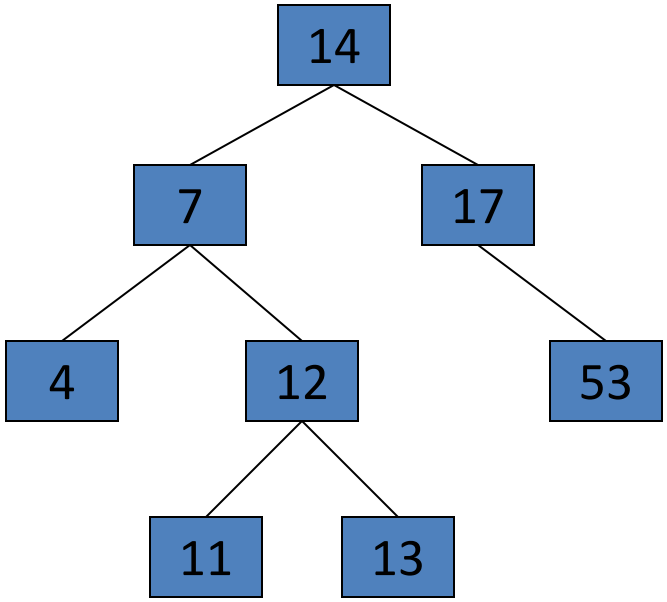
AVL Tree Example:

- **Now insert 12**



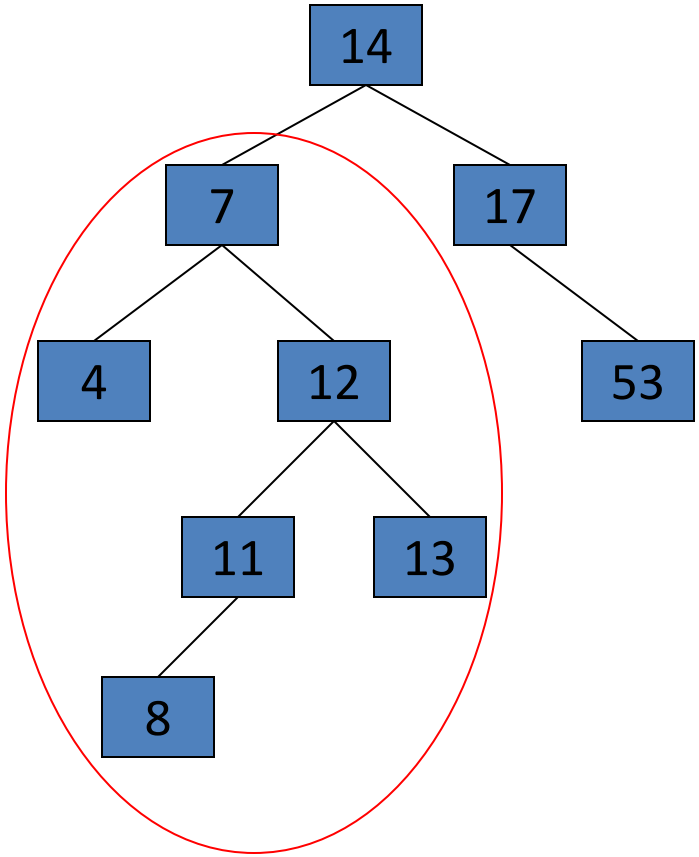
AVL Tree Example:

- **Now the AVL tree is balanced.**



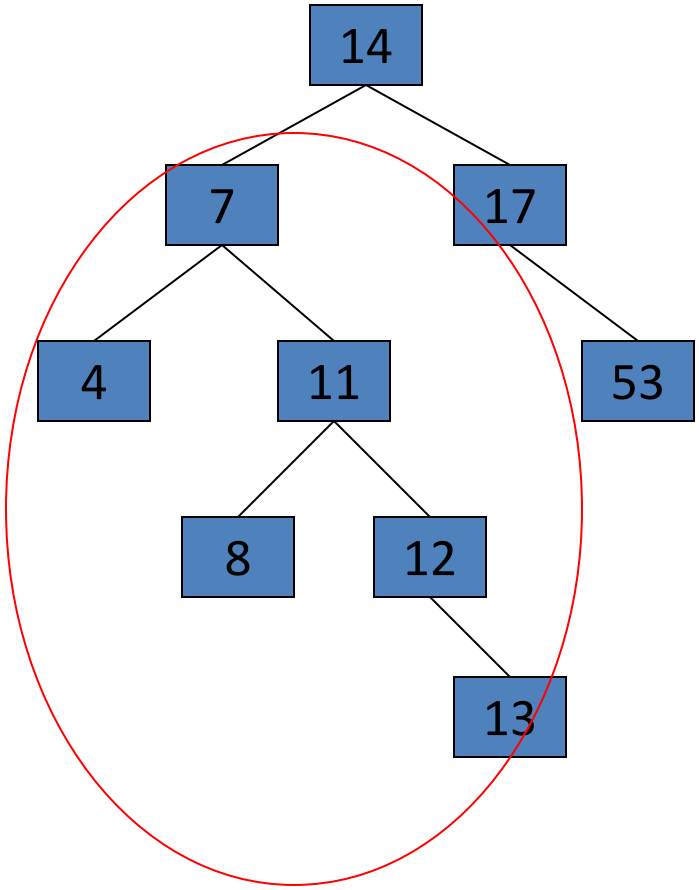
AVL Tree Example:

- **Now insert 8**



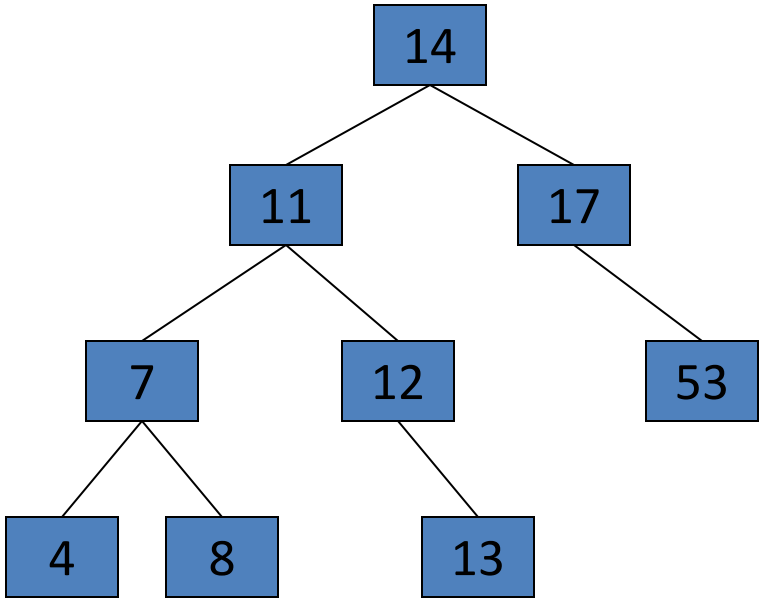
AVL Tree Example:

- **Now insert 8**



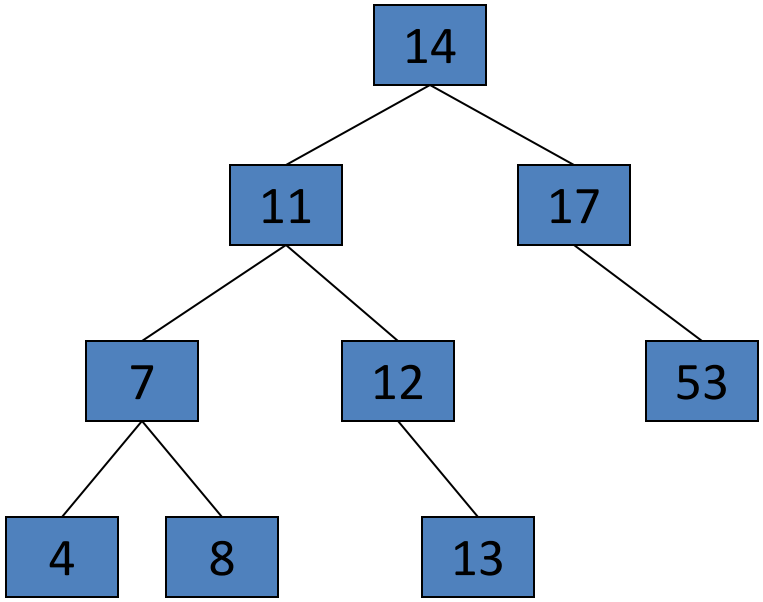
AVL Tree Example:

- **Now the AVL tree is balanced.**



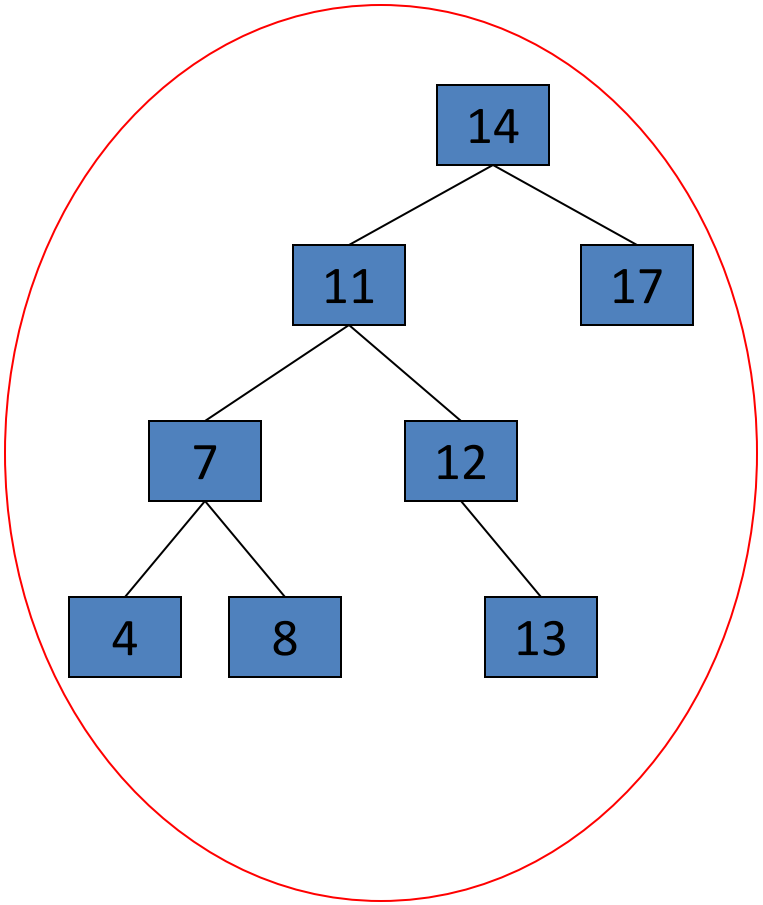
AVL Tree Example:

- **Now remove 53**



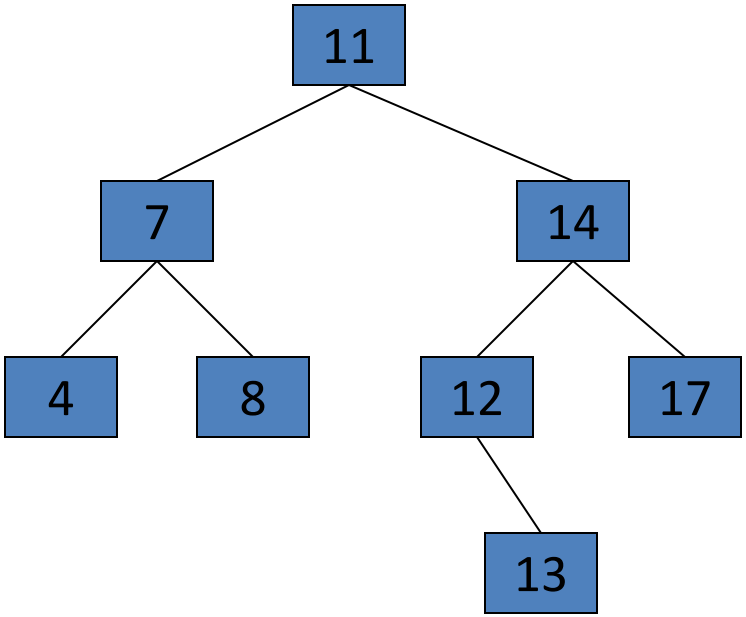
AVL Tree Example:

- **Now remove 53, unbalanced**



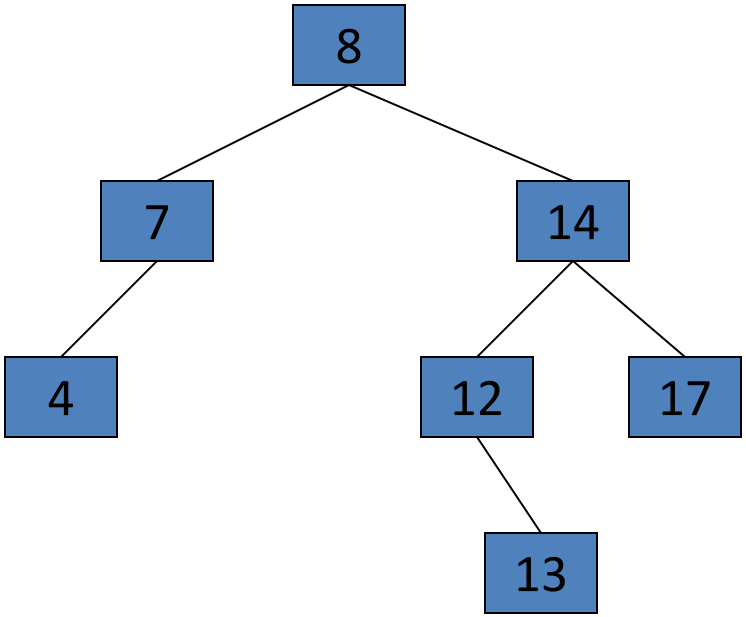
AVL Tree Example:

- **Balanced! Remove 11**



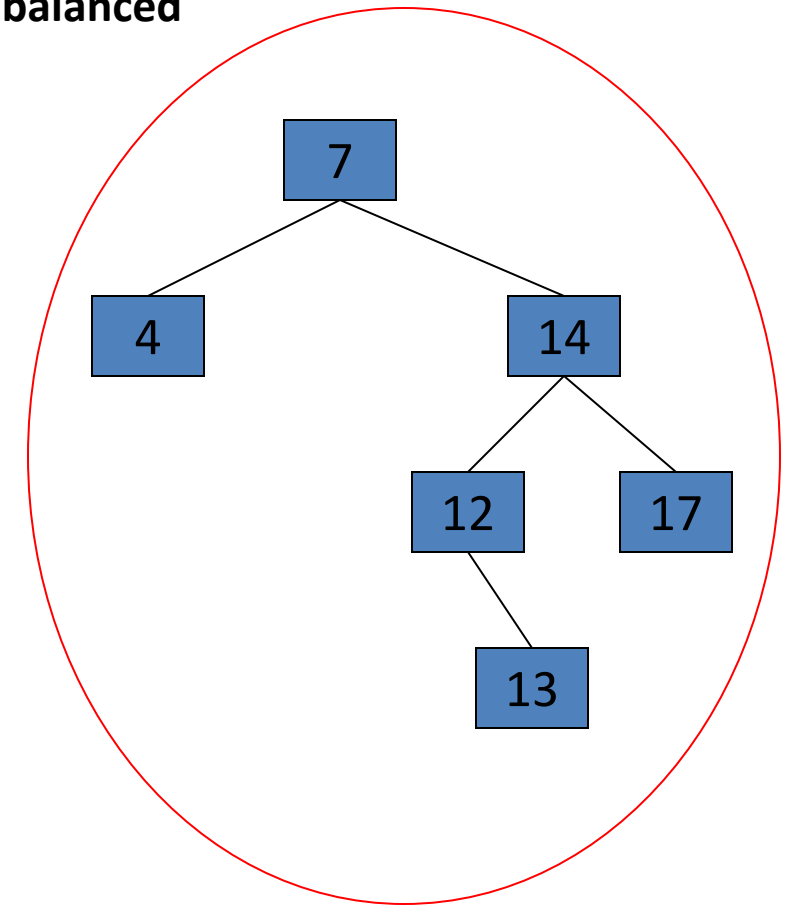
AVL Tree Example:

- **Remove 11, replace it with the largest in its left branch**



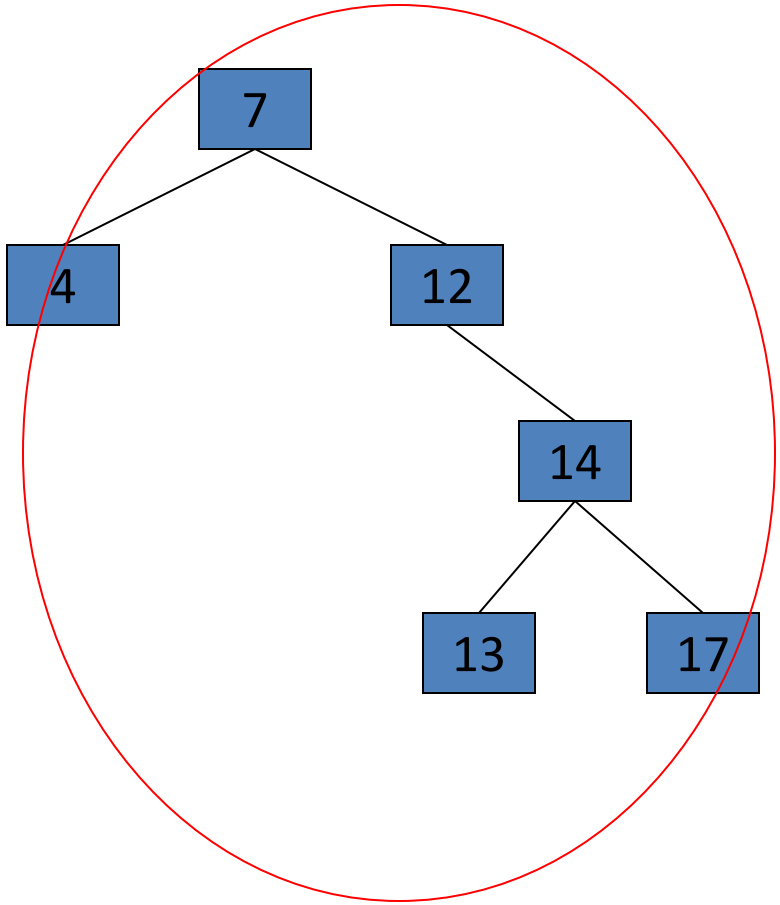
AVL Tree Example:

- **Remove 8, unbalanced**



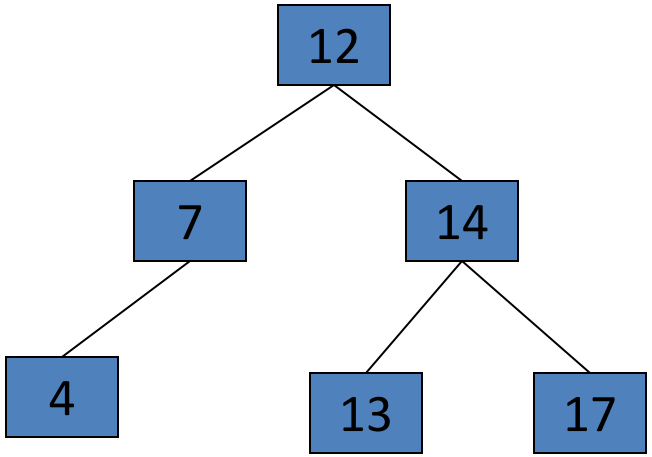
AVL Tree Example:

- **Remove 8, unbalanced**



AVL Tree Example:

- **Balanced!!**

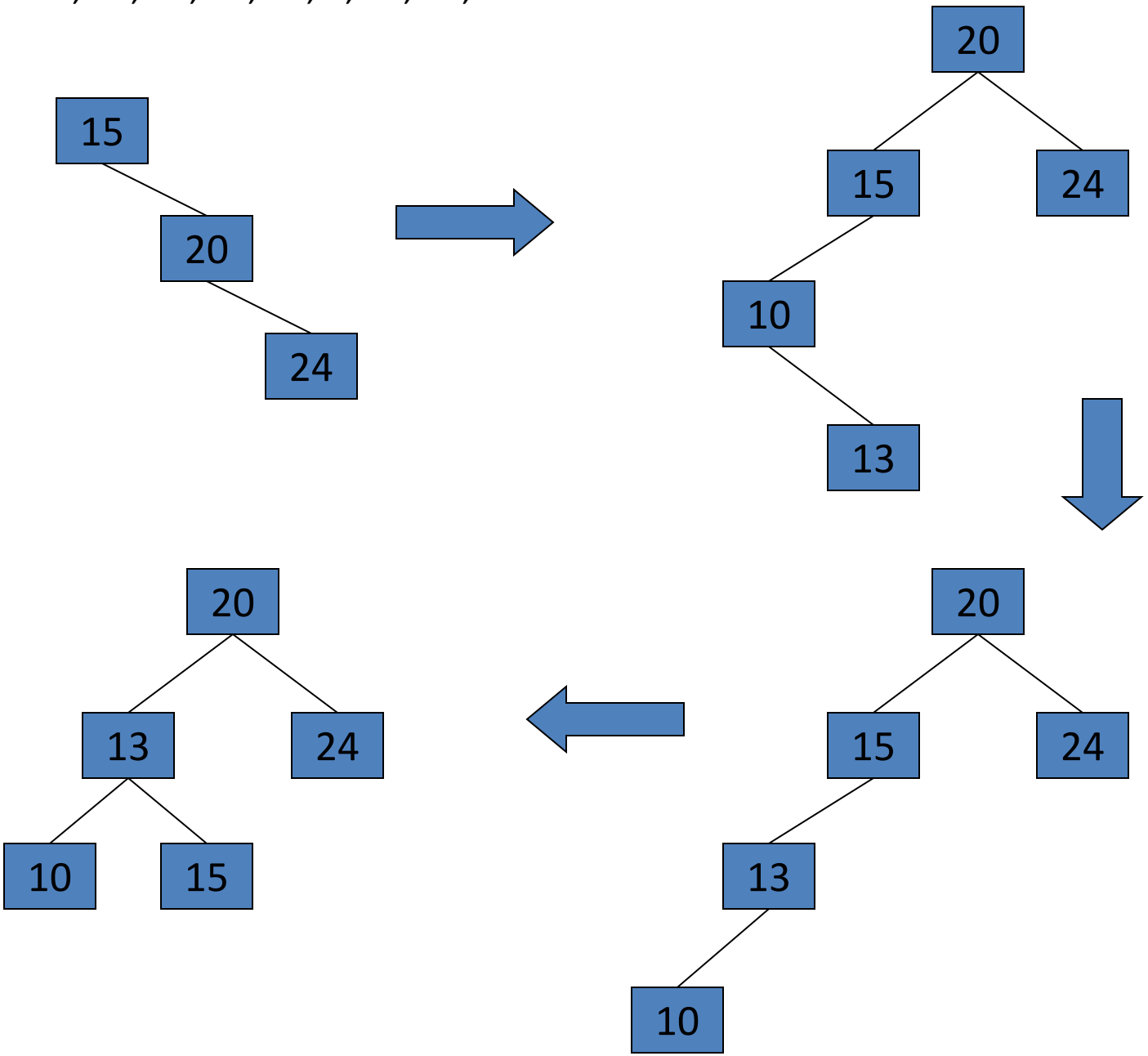


In Class Exercises

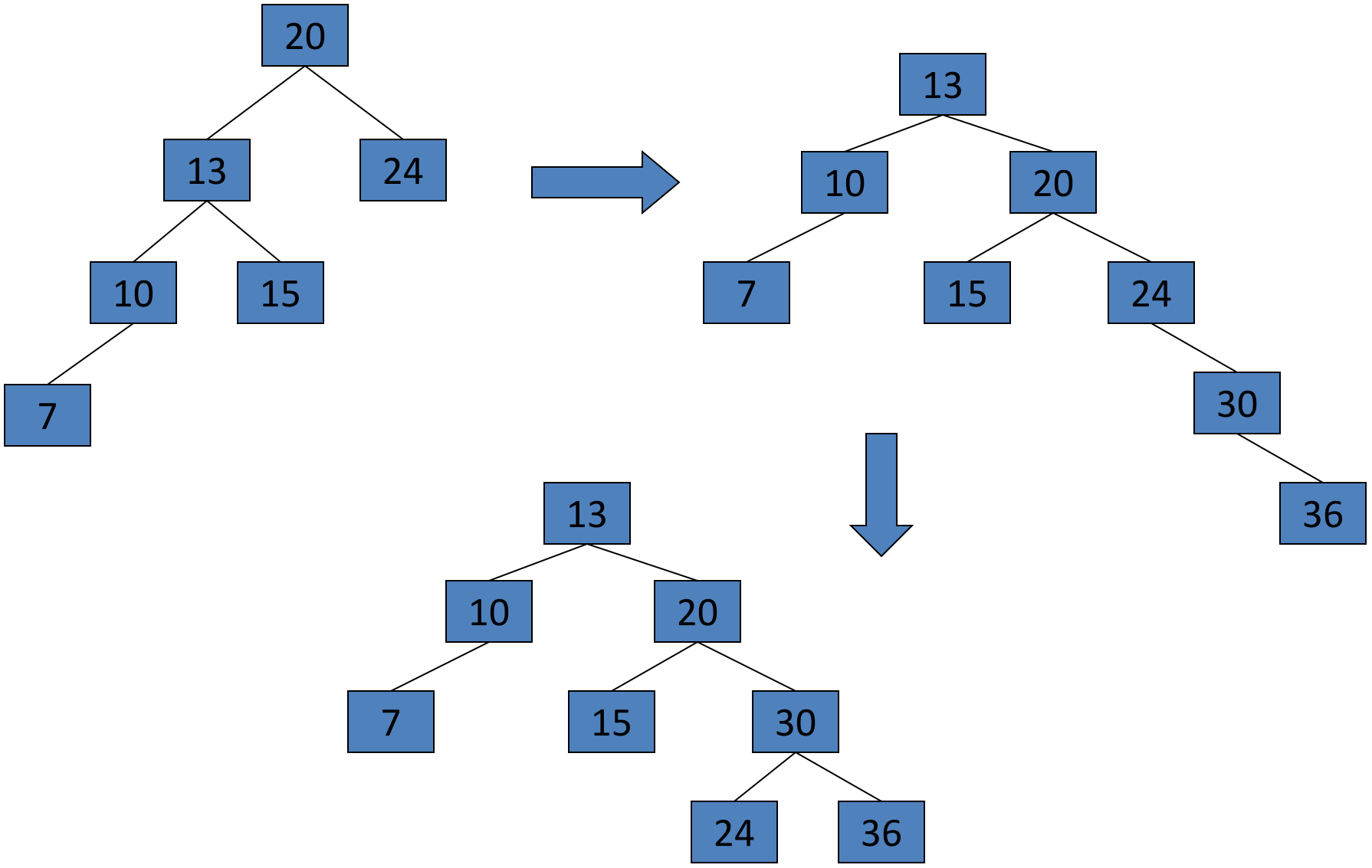
- Build an AVL tree with the following values:

  15, 20, 24, 10, 13, 7, 30, 36, 25

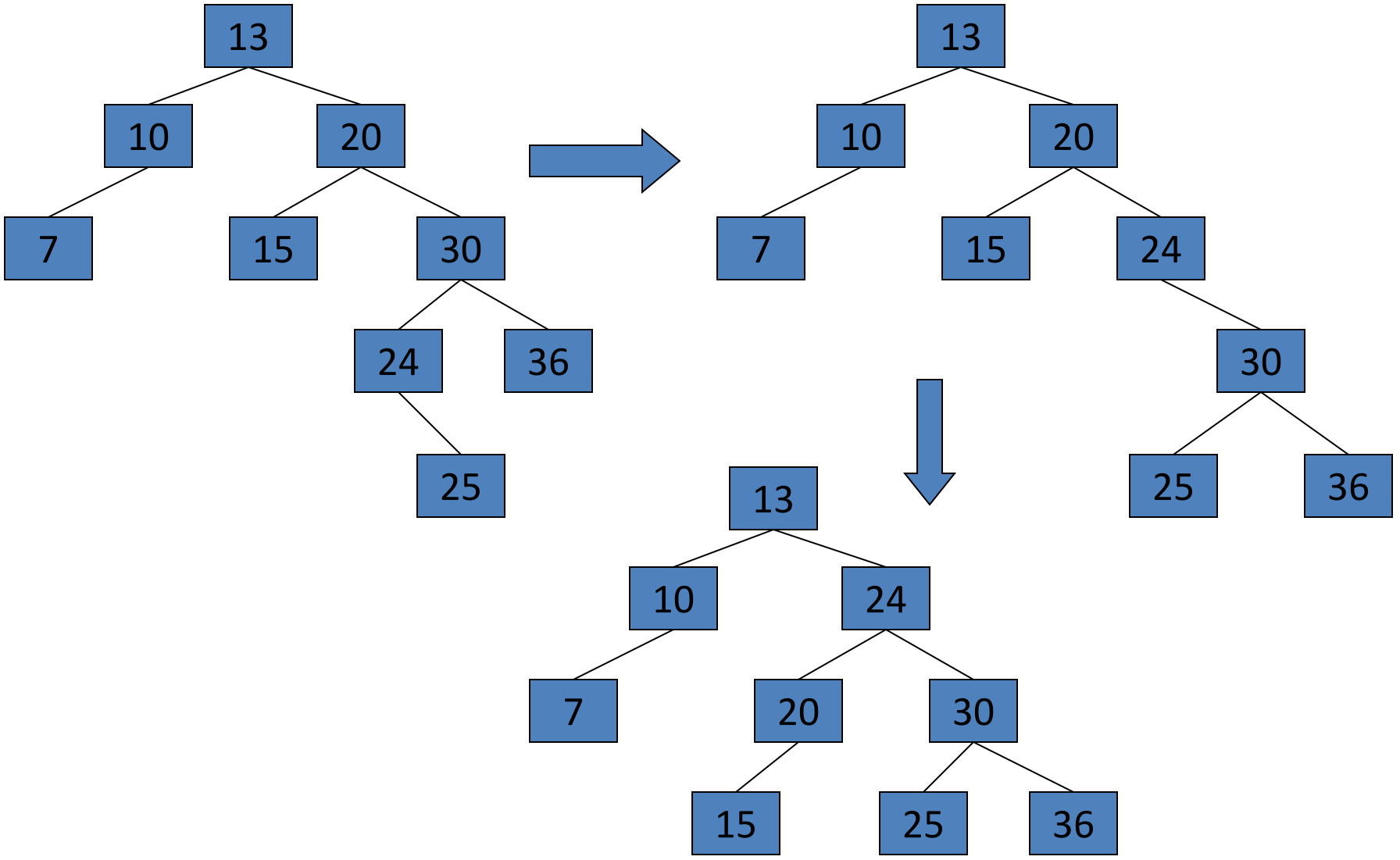
15, 20, 24, 10, 13, 7, 30, 36, 25



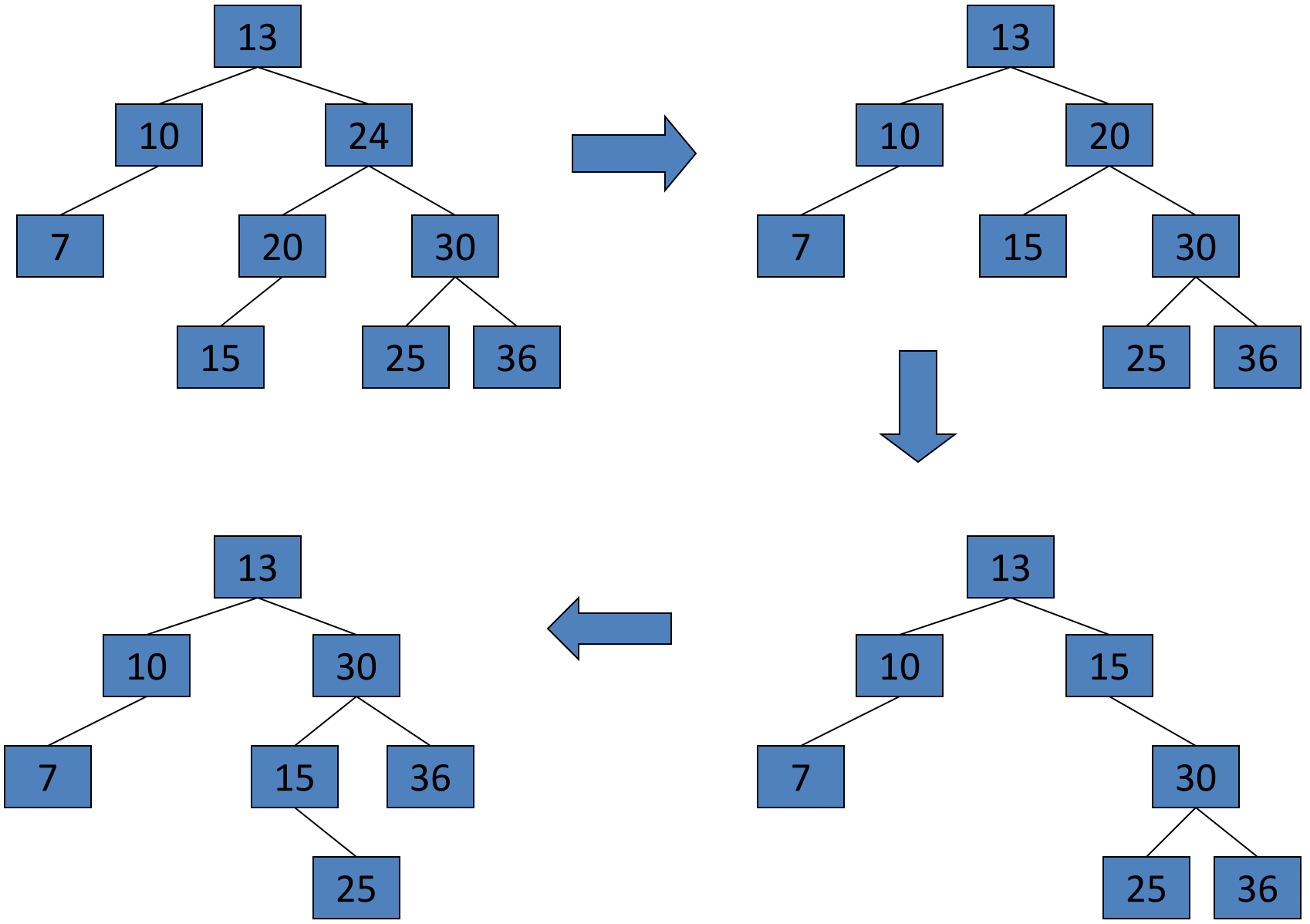
15, 20, 24, 10, 13, 7, 30, 36, 25



15, 20, 24, 10, 13, 7, 30, 36, 25



Remove 24 and 20 from the AVL tree.



Algorithm for insertion in AVL Tree

- *If the current node is NULL, create a new node with the given key.*
- *If the key is less than the current node's key, recursively insert the key in the left subtree.*
- *If the key is greater than the current node's key, recursively insert the key in the right subtree.*
- *After insertion, update the height of the current node.*
- *Calculate the balance factor of the current node.*
- *If the balance factor is greater than 1 (left-heavy), check for Left-Left (LL) or Left-Right (LR) case:*
  - ***LL Case**: If the left child's key is greater than the inserted key, perform a right rotation.*
  - ***LR Case**: If the left child's key is less than the inserted key, perform a left rotation on the left child, then a right rotation on the current node.*
- *If the balance factor is less than -1 (right-heavy), check for Right-Right (RR) or Right-Left (RL) case:*
  - ***RR Case**: If the right child's key is less than the inserted key, perform a left rotation.*
  - ***RL Case**: If the right child's key is greater than the inserted key, perform a right rotation on the right child, then a left rotation on the current node.*

Algorithm for Deletion in AVL Tree

- *If the node to be deleted is not found, return NULL.*
- *If the key to be deleted is less than the current node's key, recursively delete the key in the left subtree.*
- *If the key to be deleted is greater than the current node's key, recursively delete the key in the right subtree.*
- *If the key to be deleted is equal to the current node's key:*
  - *If the node has only one child or no child, replace the node with its child.*
  - *If the node has two children, find the in order successor (smallest in the right subtree), copy its key to the node, and delete the inorder successor.*
- *After deletion, update the height of the current node.*
- *Calculate the balance factor of the current node.*
- *Check and fix imbalance just like in insertion.*
- *Return the node after performing necessary rotations to balance the tree.*

HEAPS

What is a Heap?

- Definition: A **heap** is a special tree-based data structure that satisfies the **heap property**, and is usually implemented as a complete binary tree.
- Two types:
  - **Max-Heap**: parent ≥ children
  - **Min-Heap**: parent ≤ children
- Used to implement priority queues

Array Representation of Heaps

- How a binary heap can be stored in an array:
  - For node at index *i*
    - Left child at 2*i
    - Right child at 2*i + 1
    - Parent at (i)/2 (integer division)

Properties of Heap

- The minimum or maximum element is always at the root of the heap, allowing constant-time access.
- The relationship between a parent node at index **'i'** and its children is given by the formulas: left child at index **2i+1** and right child at index **2i+2** for 0-based indexing of node numbers.
- As the tree is complete binary, all levels are filled except possibly the last level. And the last level is filled from left to right.
- When we insert an item, we insert it at the last available slot and then rearrange the nodes so that the heap property is maintained.
- When we remove an item, we swap root with the last node to make sure either the max or min item is removed. Then we rearrange the remaining nodes to ensure heap property (max or min)

HEAPS

Heap Operations

- **Insert** (or push)
- **Extract-Min / Extract-Max**
- **Peek / Find-Min / Find-Max**
- **Build-Heap** (from an unsorted array)

Heapify (Sift-Down / Sift-Up)

- Explain *sift-up* (used during insertion): comparing child with its parent, swapping until heap property restored.
- Explain *sift-down* (used during extraction or in building heap): swap with appropriate child and continue down.

Algorithm for Min heap

INSERTION:

1. Insert an element in a heap tree
2. Compare the value of child node with its parent
3. If the value of parent is greater than child, then swap them
4. Repeat step 2 & 3 until heap property satisfies

DELETION:

1. Delete the root node
2. Move the last element of last level to root
3. Compare the value of child node with its parent
4. If the value of parent is greater than child, then swap them
5. Repeat step 3 & 4 until heap property satisfies

Algorithm for Max heap

INSERTION:

1. Insert an element in a heap tree
2. Compare the value of child node with its parent
3. If the value of parent is Lesser than child, then swap them
4. Repeat step 2 & 3 until heap property satisfies

DELETION:

1. Delete the root node
2. Move the last element of last level to root
3. Compare the value of child node with its parent
4. If the value of parent is lesser than child, then swap them
5. Repeat step 3 & 4 until heap property satisfies

Routine for MAX heap

```
Insert(A, n, value)
{ n=n+1;
  A[n]=value;
  i=n;
While(i>1)
{ parent =floor(i/2);
If(A[parent]>=A[i])
   swap(A[parent],A[i]);
Else
   return;
}}
```

Routine for MIN heap

```
Insert(A, n, value)
{ n=n+1;
  A[n]=value;
  i=n;
While(i>1)
{ parent =floor(i/2);
If(A[parent]<=A[i])
   swap(A[parent],A[i]);
Else
   return;
}}
```

Applications of Heaps

- **priority Queues**: Heaps are commonly used to implement priority queues, where elements with higher priority are extracted first. This is useful in many applications such as scheduling tasks, handling interruptions, and processing events.
- **Sorting Algorithms**: Heapsort, a comparison-based sorting algorithm, is implemented using the Heap data structure. It has a time complexity of O(n log n), making it efficient for large datasets.
- **Graph algorithms**: Heaps are used in graph algorithms such as Prim's Algorithm, Dijkstra's algorithm., and the A* search algorithm.
- **Lossless Compression**: Heaps are used in data compression algorithms such as Huffman coding, which uses a priority queue implemented as a min-heap to build a Huffman tree.
- **Medical Applications**: In medical applications, heaps are used to store and manage patient information based on priority, such as vital signs, treatments, and test results.

Applications of Heaps

- **Load balancing**: Heaps are used in load balancing algorithms to distribute tasks or requests to servers, by processing elements with the lowest load first.
- ***Order statistics****:* The Heap data structure can be used to efficiently find the kth smallest (or largest) element in an array. See method 4 and 6 of this post for details.
- **Resource allocation:** Heaps can be used to efficiently allocate resources in a system, such as memory blocks or CPU time, by assigning a priority to each resource and processing requests in order of priority.
- **Job scheduling**: The heap data structure is used in job scheduling algorithms, where tasks are scheduled based on their priority or deadline. The heap data structure allows efficient access to the highest-priority task, making it a useful data structure for job scheduling applications